

Lecture 4

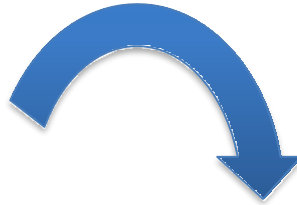
Problem Solving as Search

[Blind / Uninformed vs. Heuristic / Informed Strategies]

Resources for this lecture

- This lecture covers the following chapters/sections:
 - Chapter 3 (Structures & Strategies for State Space Search; sections 3.2, and 3.3) and Chapter 4 (Heuristic Search) *from* George F. Luger, "Artificial Intelligence: Structures and strategies for complex problem solving", Sixth edition (2009), Pearson Education Limited.

Outline



- **Blind vs. Heuristic Strategies**
 - **Blind Strategies**
 - **Depth-First Strategy**
 - **Depth-Limited Strategy**
 - **Iterative-Deepening Strategy**
 - **Breadth-First Strategy**
 - **Bidirectional Search Strategy**
 - **Comparison of Blind Search Strategies**
 - **Complexity of Basic Search Algorithms**
 - **Repeated States**
 - **Avoiding Repeated States**
 - **Uniform-Cost Strategy**
 - **Best-First Search**
- **Heuristic Function**
 - **Robot Navigation**
 - **Examples of Evaluation function**
 - **8-Puzzle**
- **More on Heuristic Search & Functions**
 - **Symmetry Reduction**
 - **Heuristic Reduction**
 - **Hill Climbing Strategy**
 - **Best-First Search**
 - **8-Puzzle Heuristics**

Recap: Search tree

“What if” tree of sequences of actions and outcomes

- ❖ When we are searching, we are not acting in the world, merely “thinking” about the possibilities

The root node corresponds to the starting state

The children of a node correspond to the **successor states** of that node's state

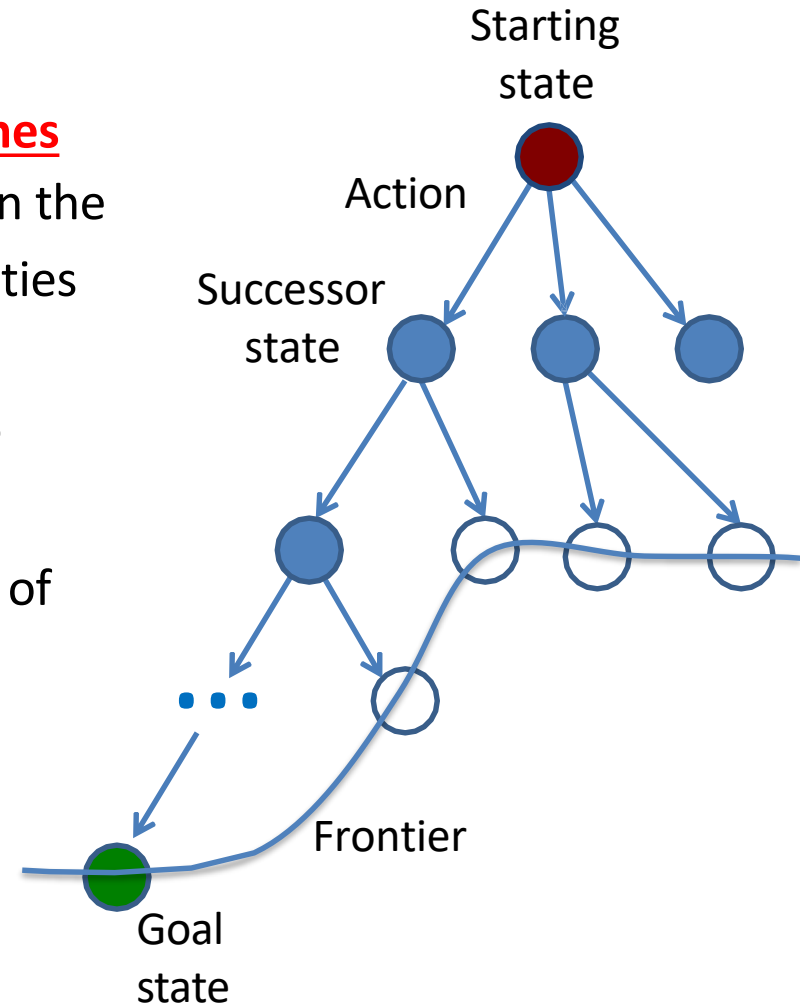
A path through the tree corresponds to a sequence of actions

- ❖ A solution is a path ending in the goal state

Nodes vs. states

A state is a representation of the world,
while a node is a data structure that is
part of the search tree

Node has to keep pointer to parent, path cost, possibly other info



Recap: Search

Given:

Initial state

Actions

Transition model

Goal state

Path cost

How do we find the optimal solution?

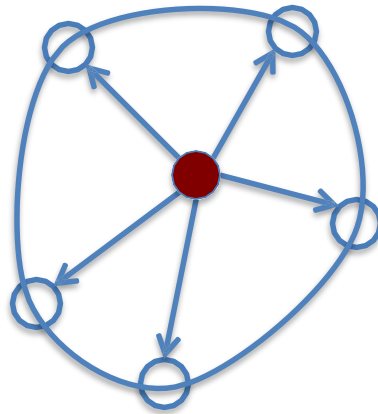
Recap: Search: Basic idea

- Let's begin at the start state and **expand** it by making a list of all possible successor states.
- Maintain a **frontier** or a list of unexpanded states.
- At each step, pick a state from the frontier to expand.
- Keep going until you reach a goal state.
- Try to expand as few states as possible.

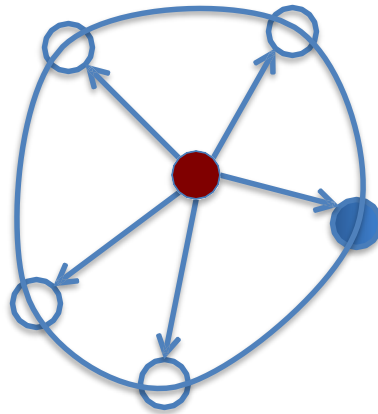
Recap: Search: Basic idea



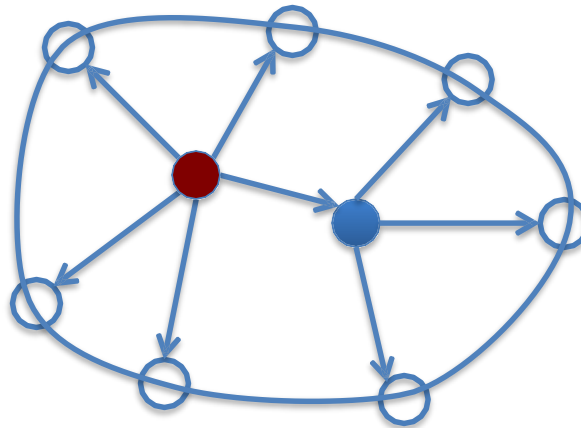
Recap: Search: Basic idea



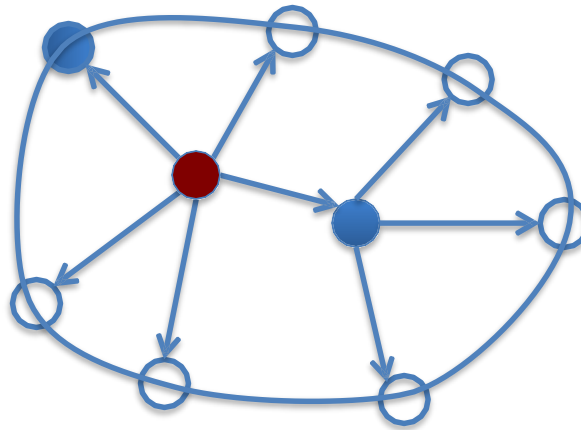
Recap: Search: Basic idea



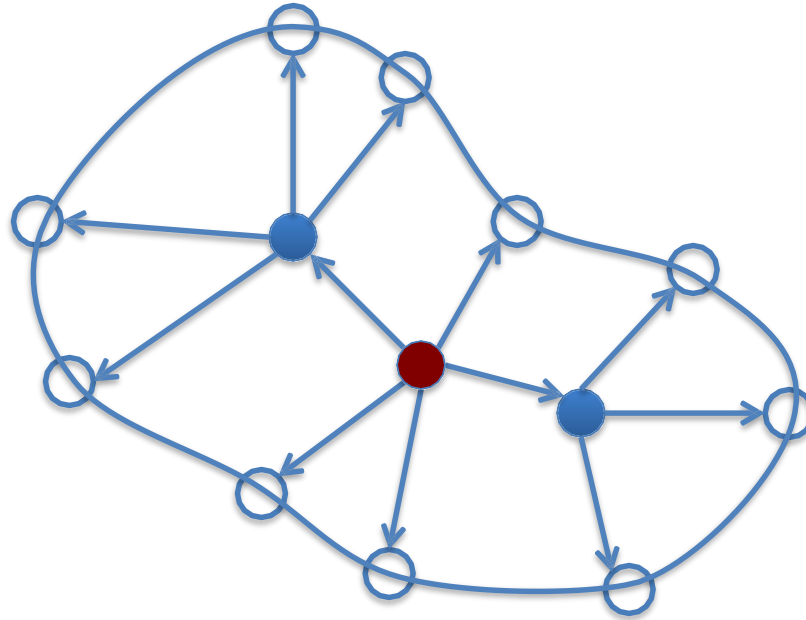
Recap: Search: Basic idea



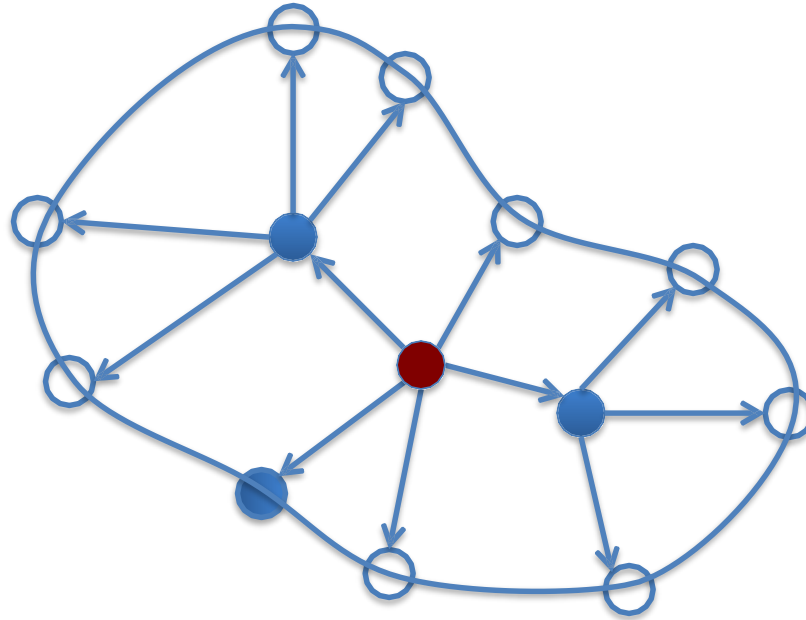
Recap: Search: Basic idea



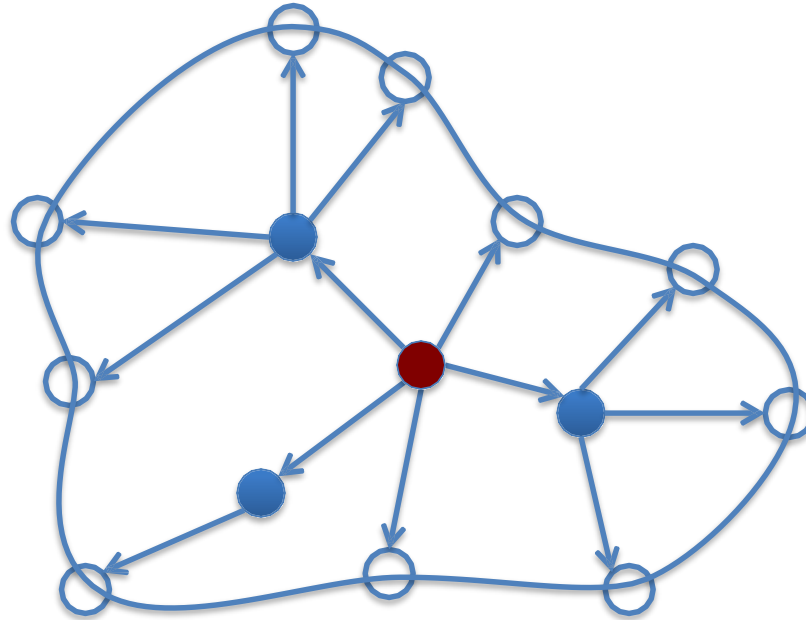
Recap: Search: Basic idea



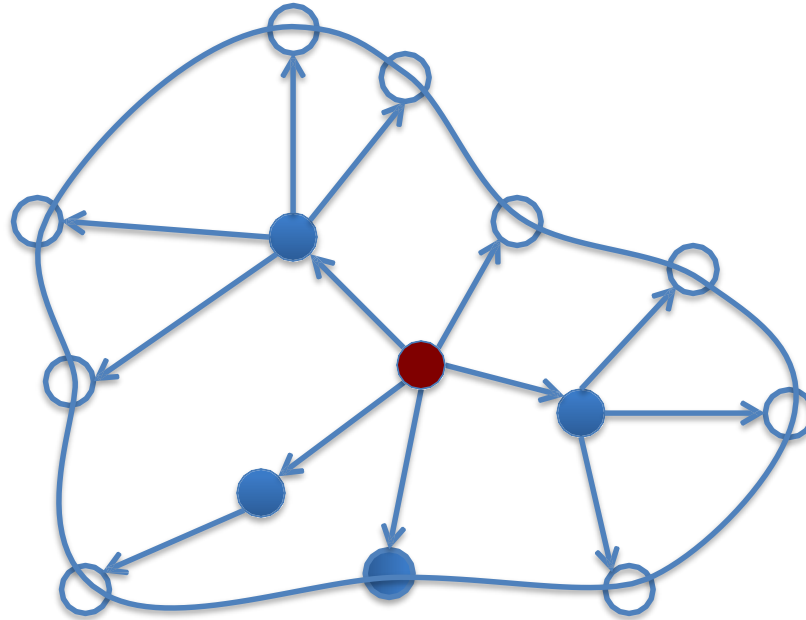
Recap: Search: Basic idea



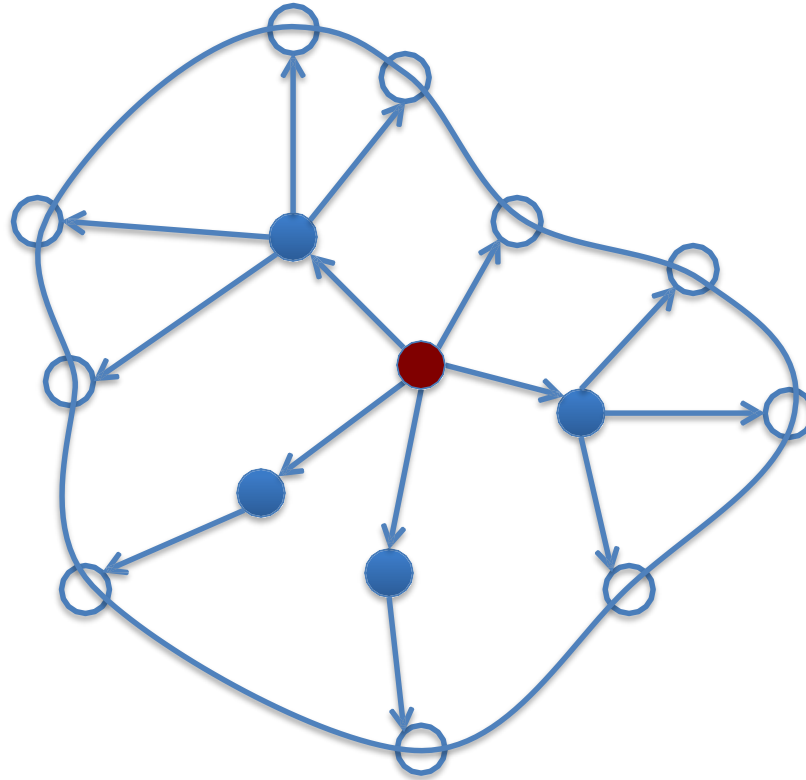
Recap: Search: Basic idea



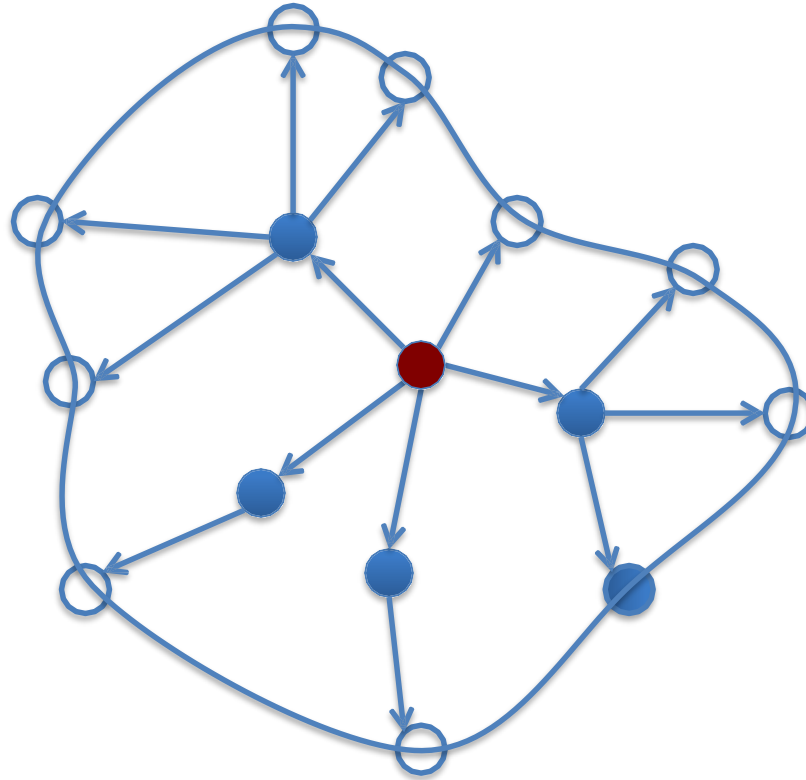
Recap: Search: Basic idea



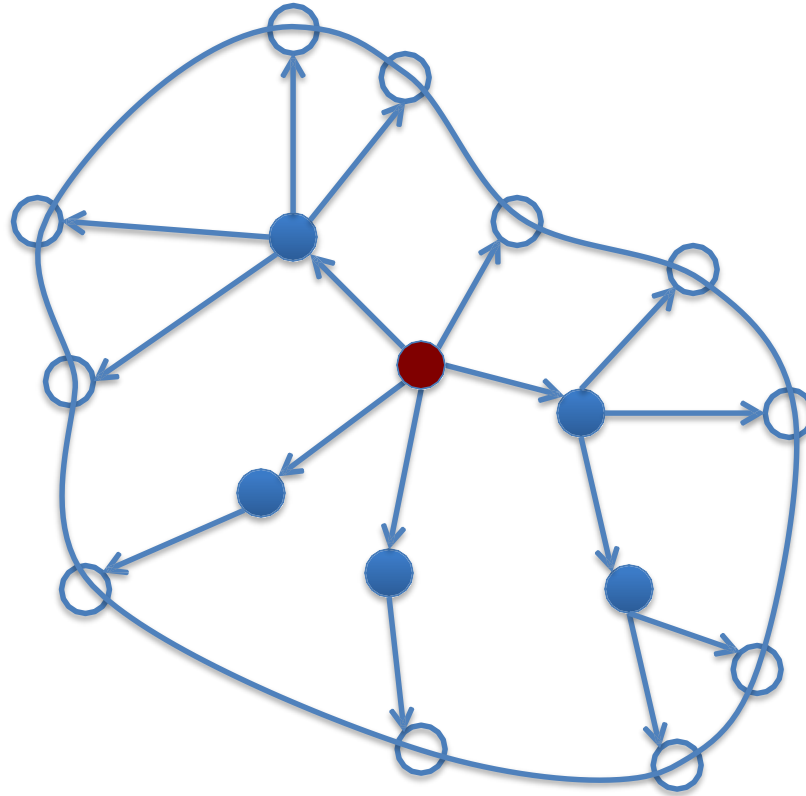
Recap: Search: Basic idea



Recap: Search: Basic idea



Recap: Search: Basic idea



Recap: Tree Search Algorithm Outline

Initialize the **frontier** using the **starting state**.

While the frontier is not empty:

- Choose a frontier node according to **search strategy** and take it off the frontier.
- If the node contains the **goal state**, return solution.
- Else **expand** the node and add its children to the frontier.

Blind vs. Heuristic Strategies

- ❖ **Blind (or Un-Informed / Exhaustive / Brute-Force)** strategies do not exploit any of the information contained in a state.
- ❖ **Heuristic (or Informed)** strategies exploits such information to assess that one node is “more promising” than another.

Uninformed Search
[**Blind Search**]
[**Exhaustive Search**]
[**Brute-Force Search**]

Blind Strategies

Breadth-First Search [BFS]

Bidirectional Search

Depth-First Search [DFS]

Depth-Limited Search

Iterative Deepening Search

Step cost = 1

A green rectangular box containing the text 'Step cost = 1'. Two green arrows originate from the left side of this box: one points to the word 'Bidirectional' in the 'Bidirectional Search' line, and the other points to the word 'DFS' in the 'Depth-First Search [DFS]' line.

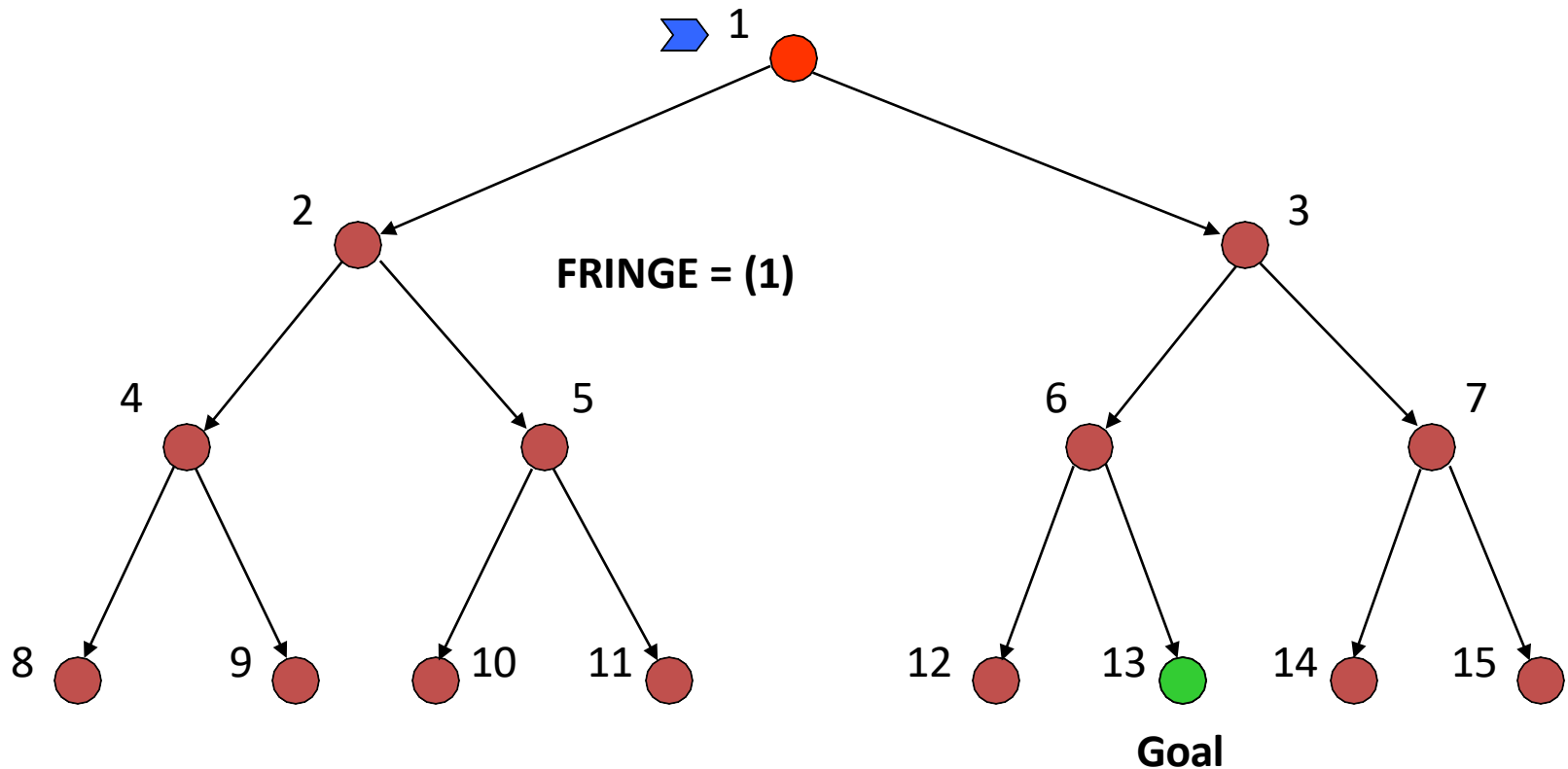
Uniform-Cost Search [UCS]

Step cost = $c(\text{action}) \geq \epsilon > 0$

A pink rectangular box containing the text 'Step cost = c(action) ≥ ε > 0'. A pink arrow originates from the left side of this box and points to the word 'Uniform' in the 'Uniform-Cost Search [UCS]' line.

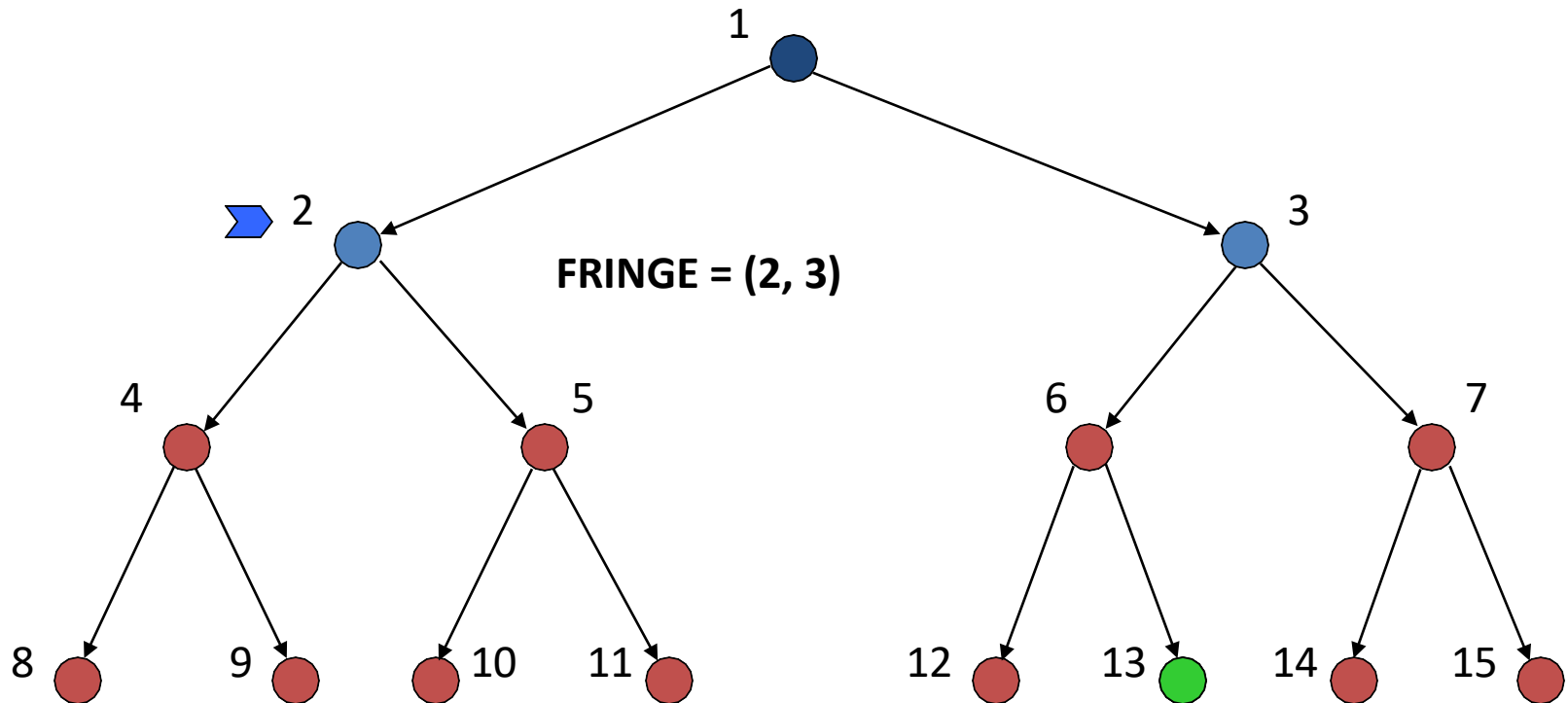
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



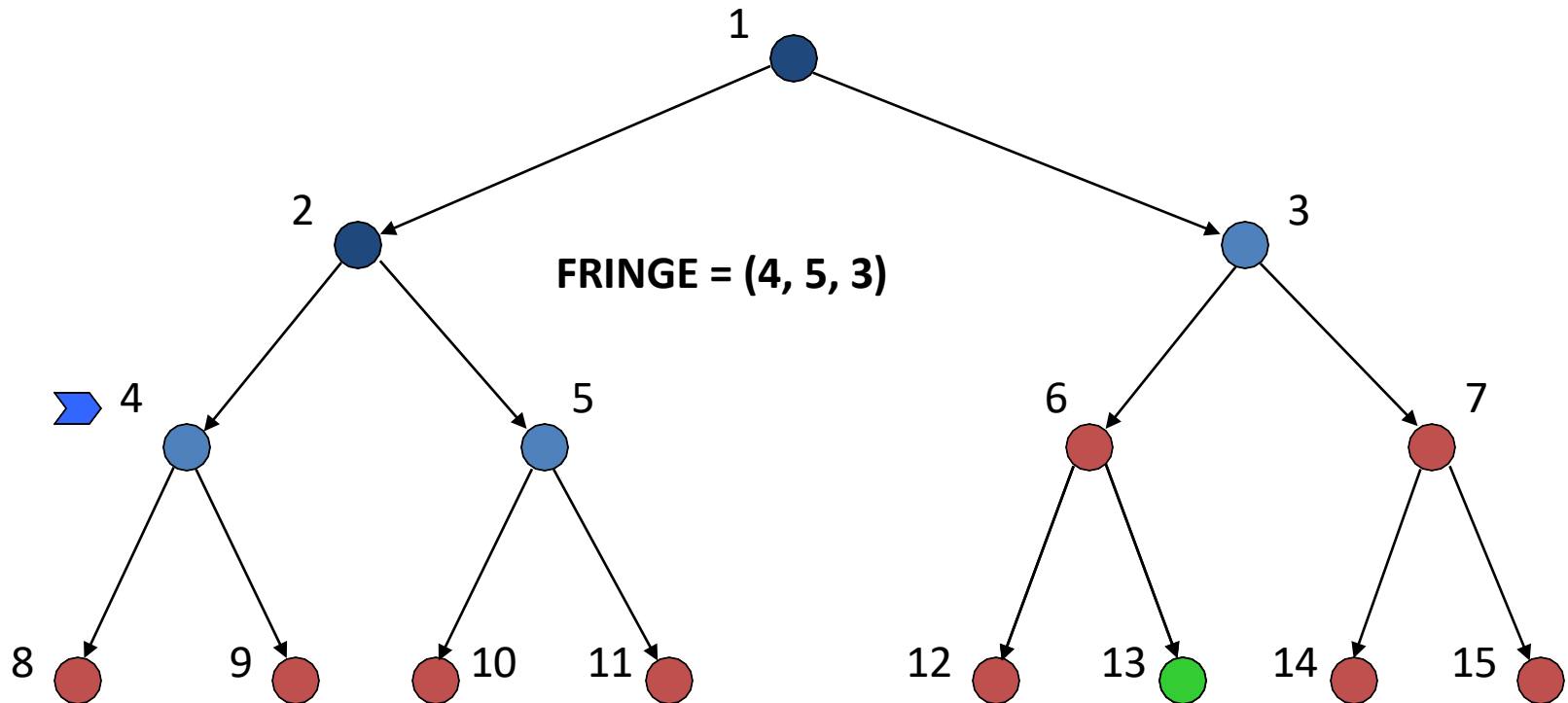
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



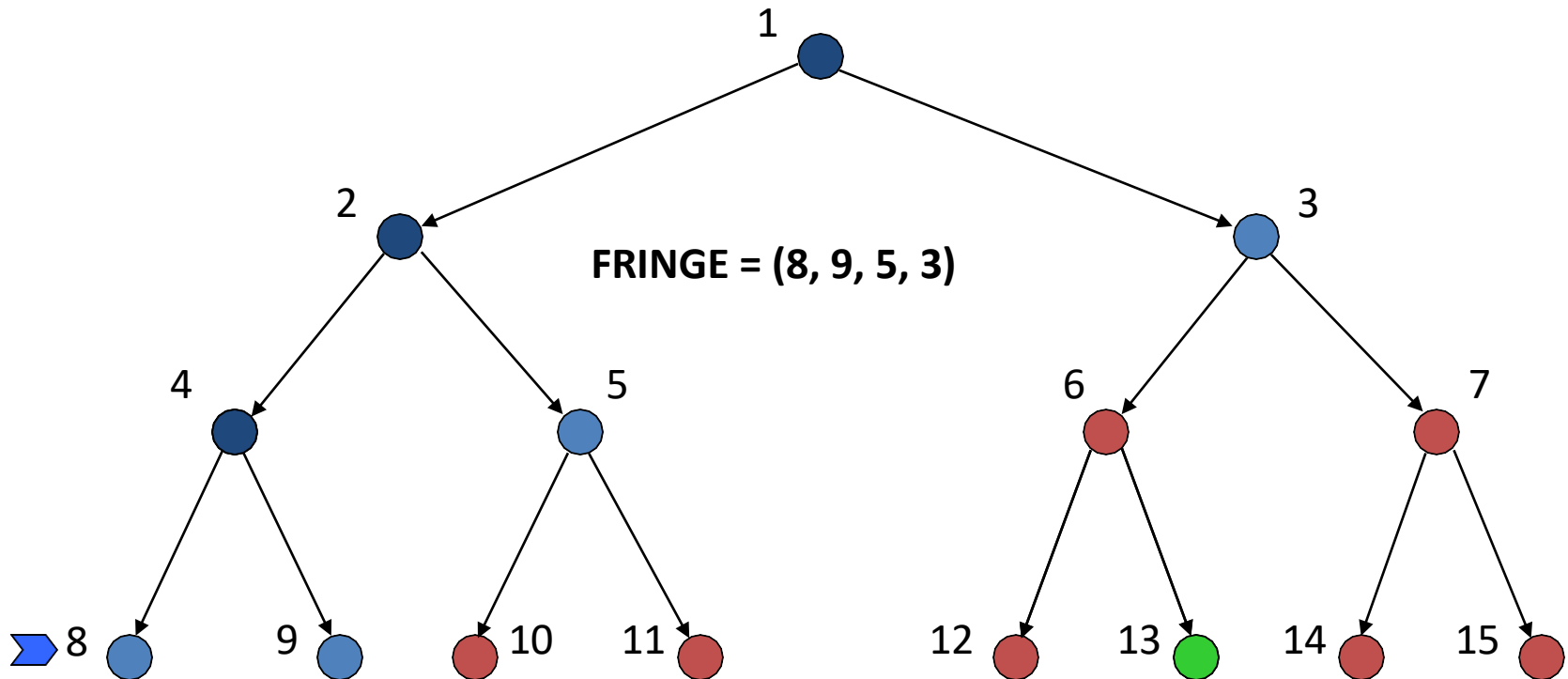
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



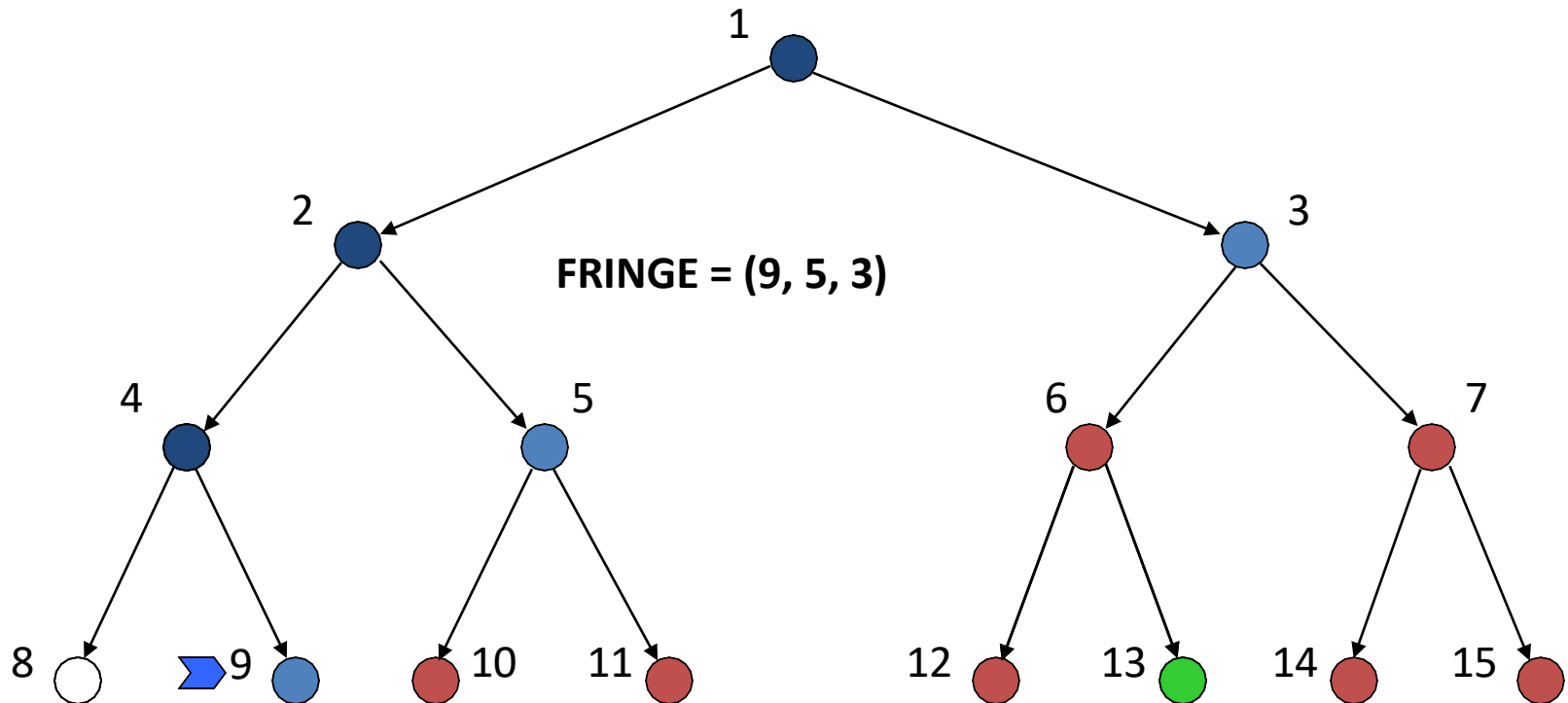
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



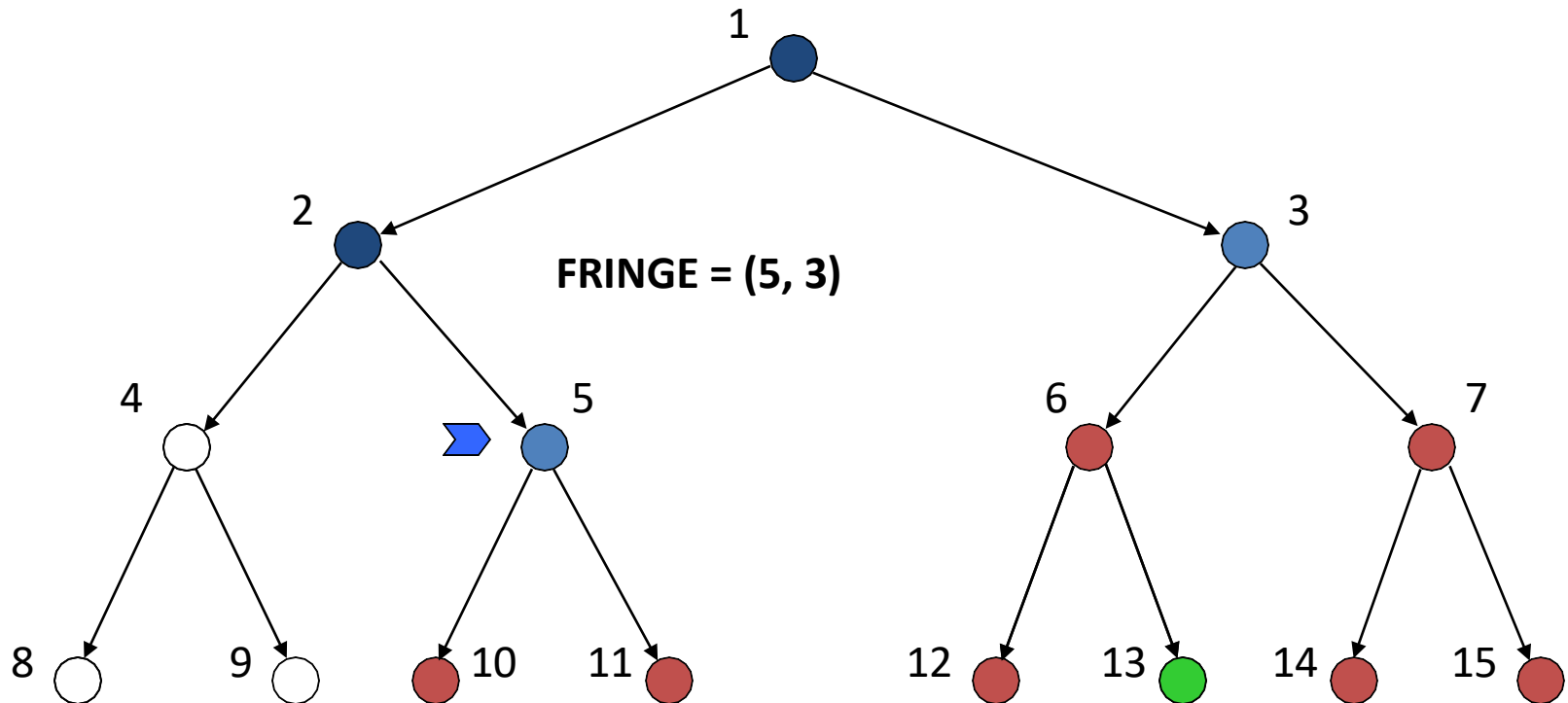
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



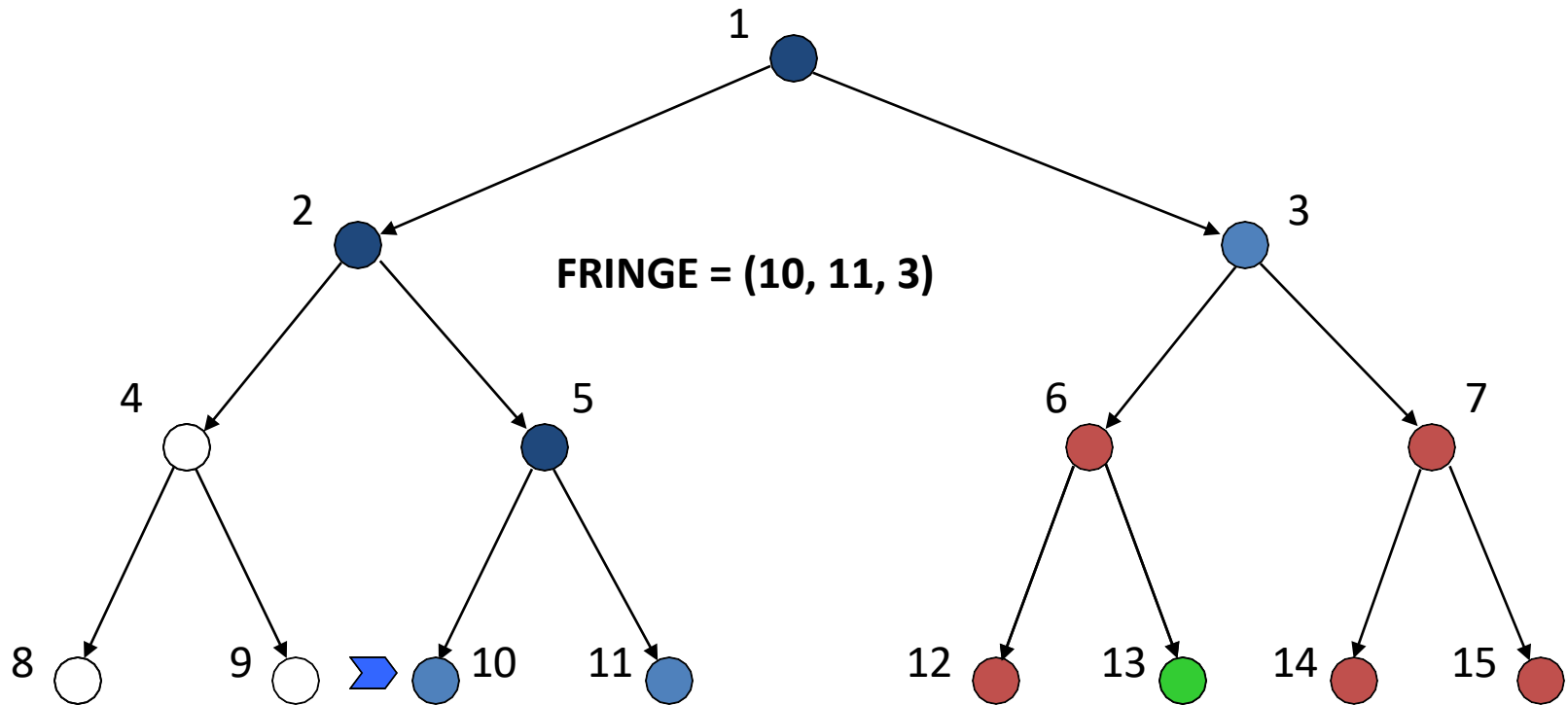
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



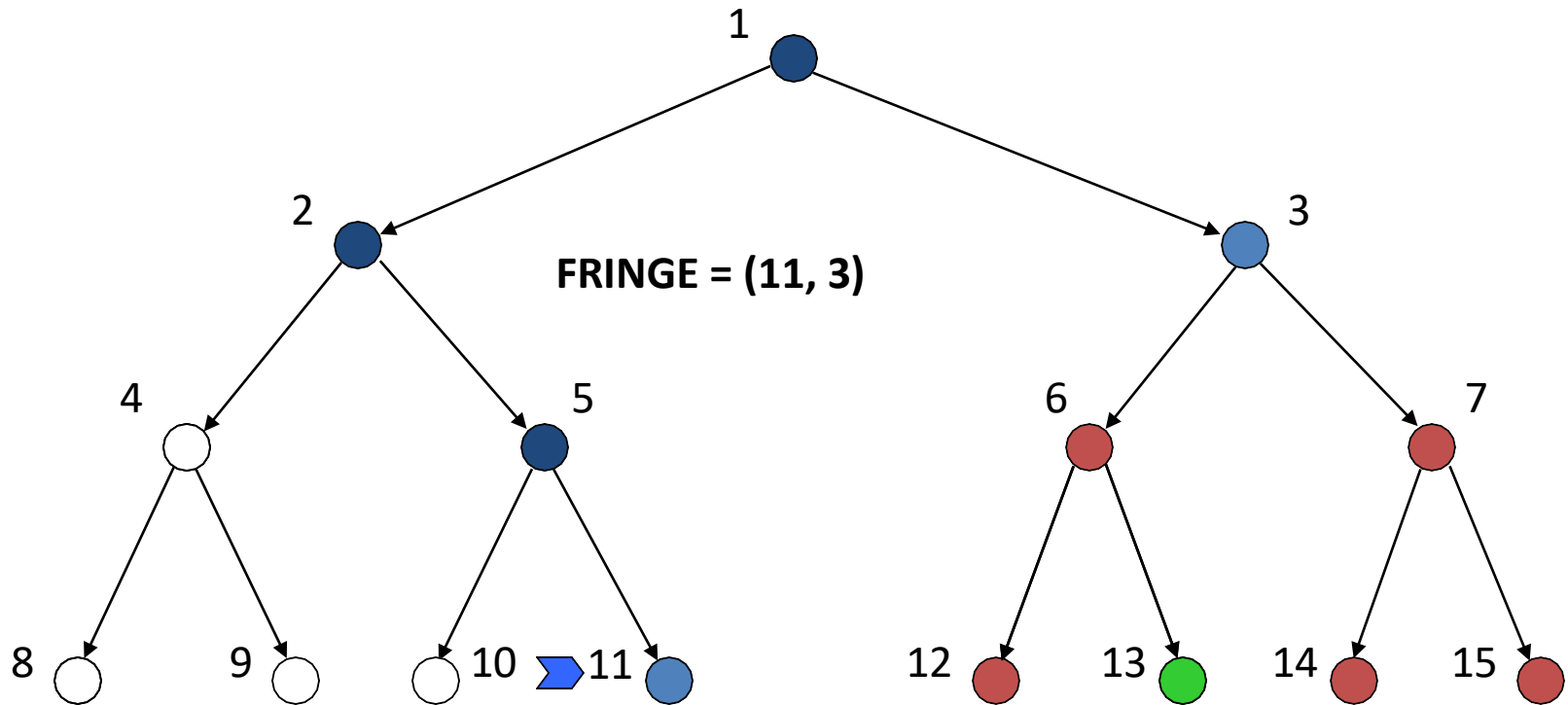
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



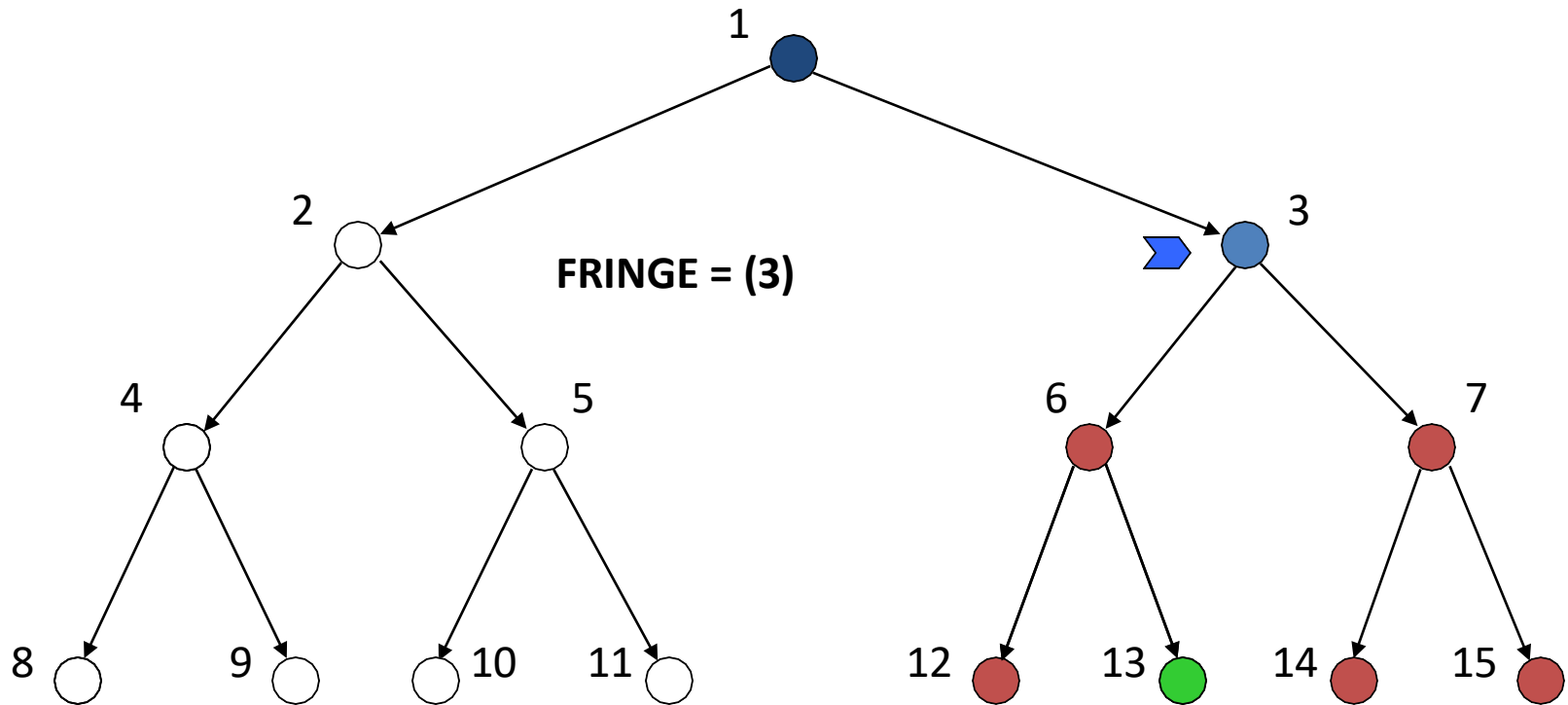
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



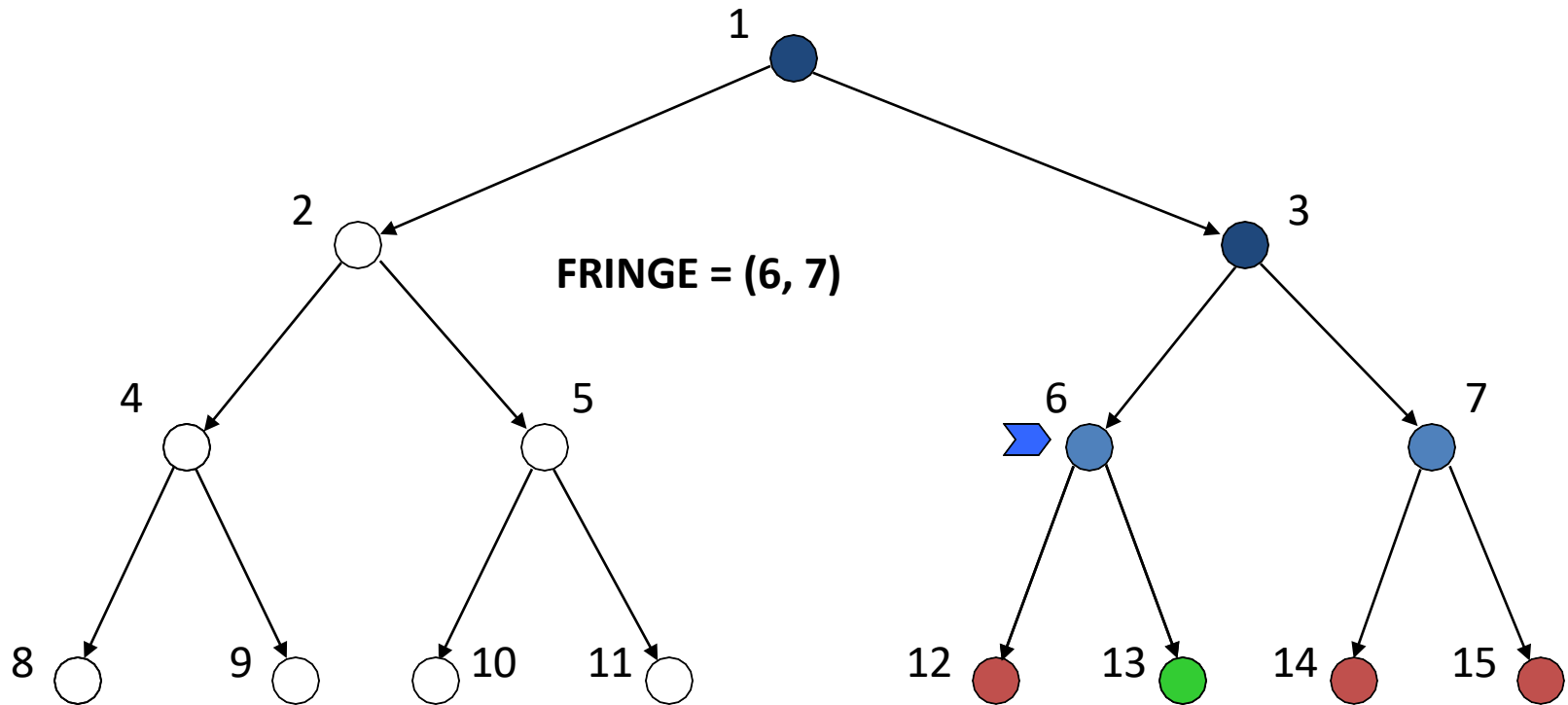
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



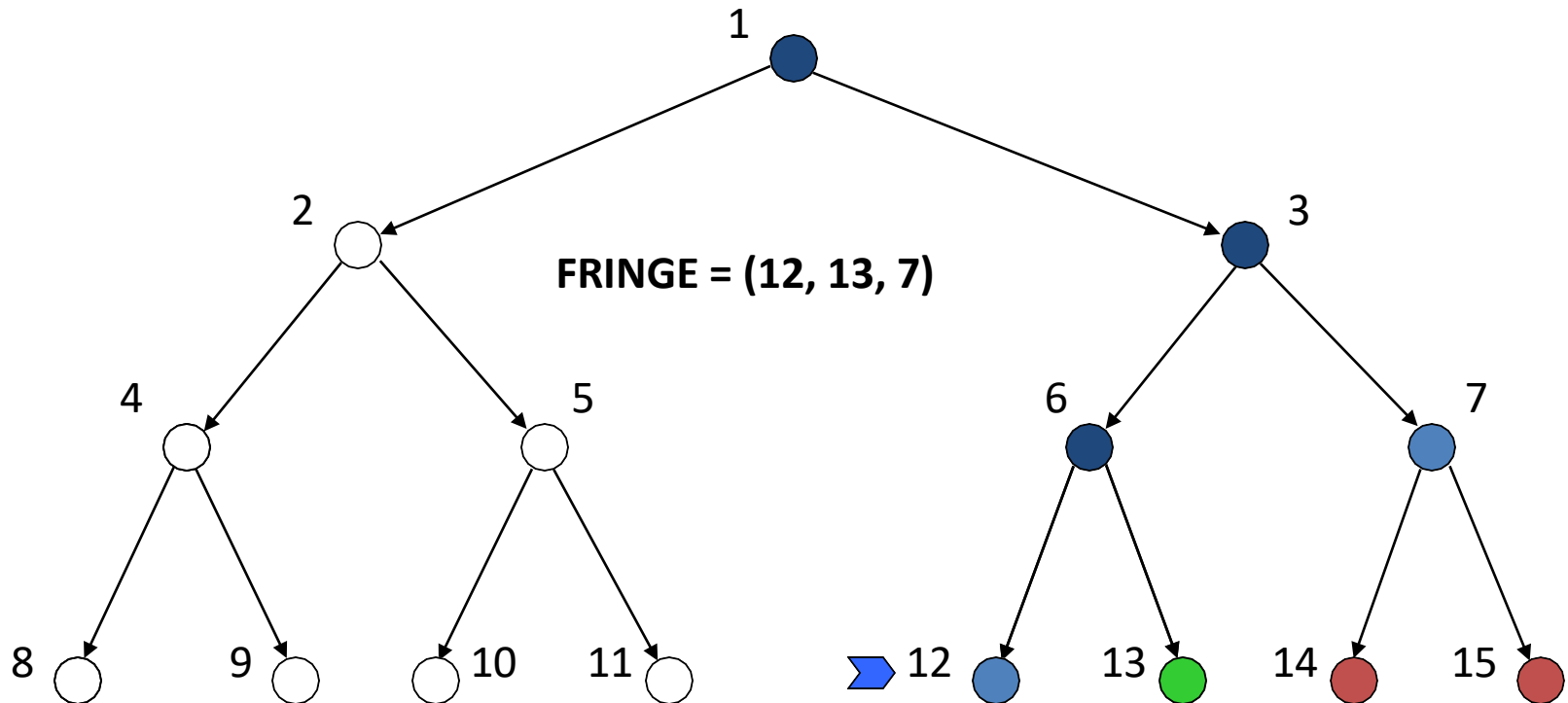
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



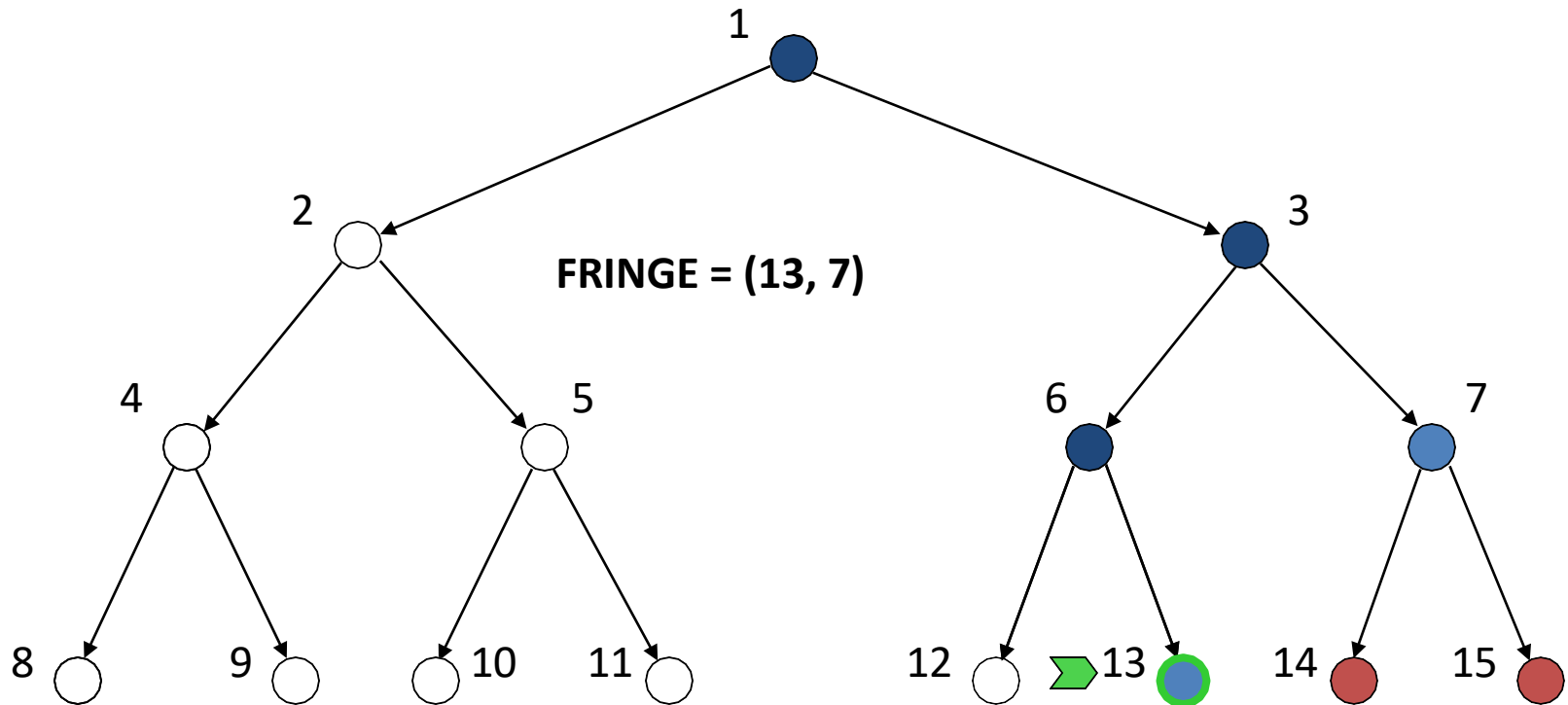
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



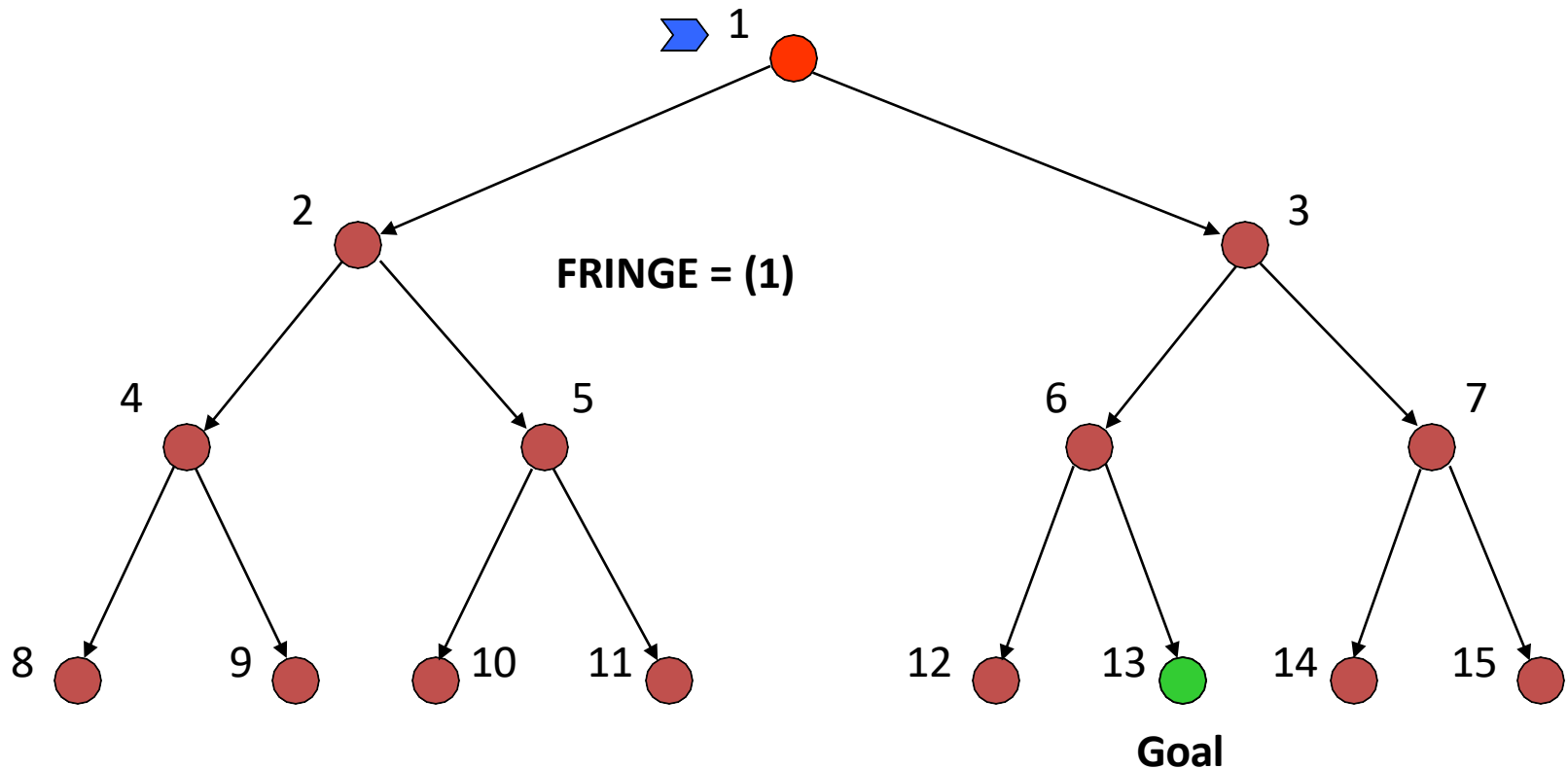
Depth-First Strategy

New nodes are inserted **at the front** of the FRINGE.



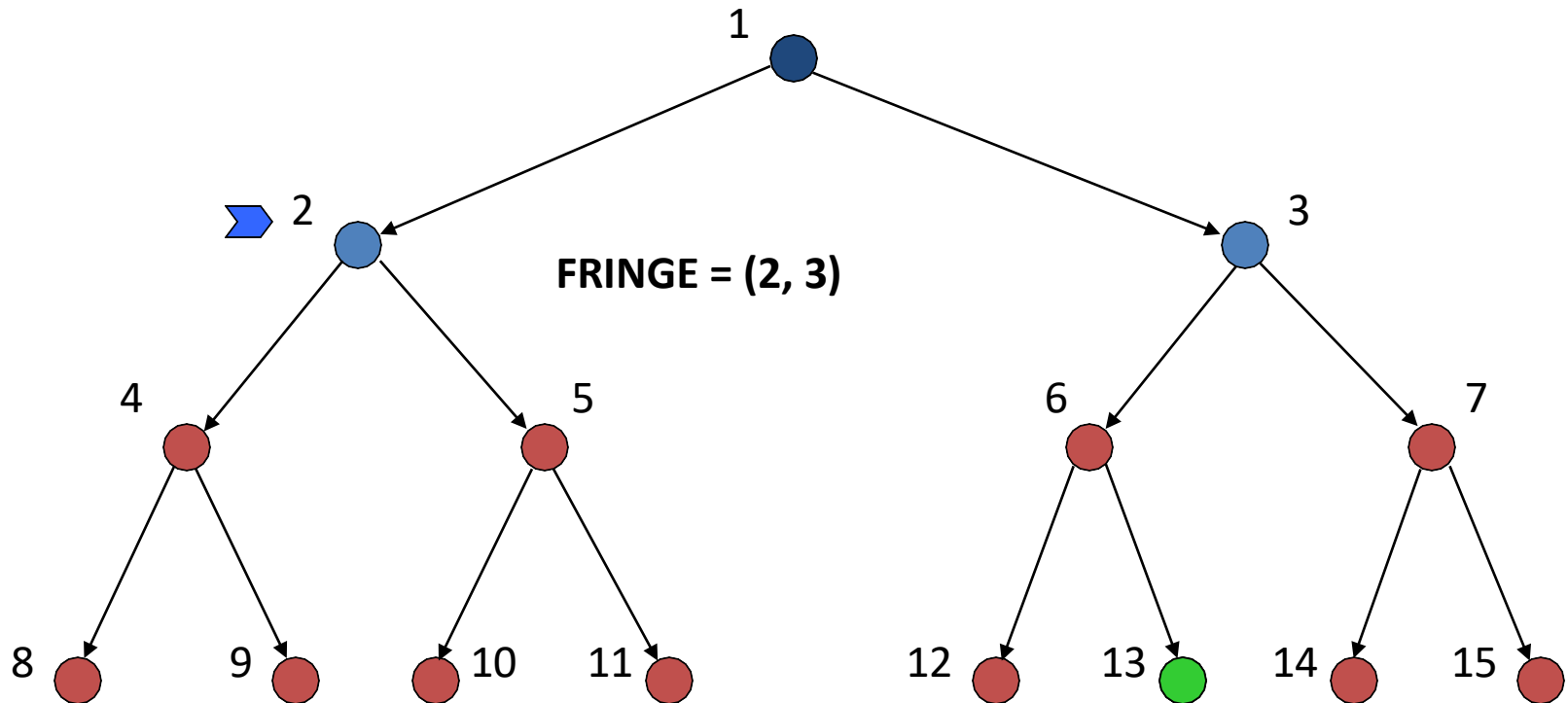
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



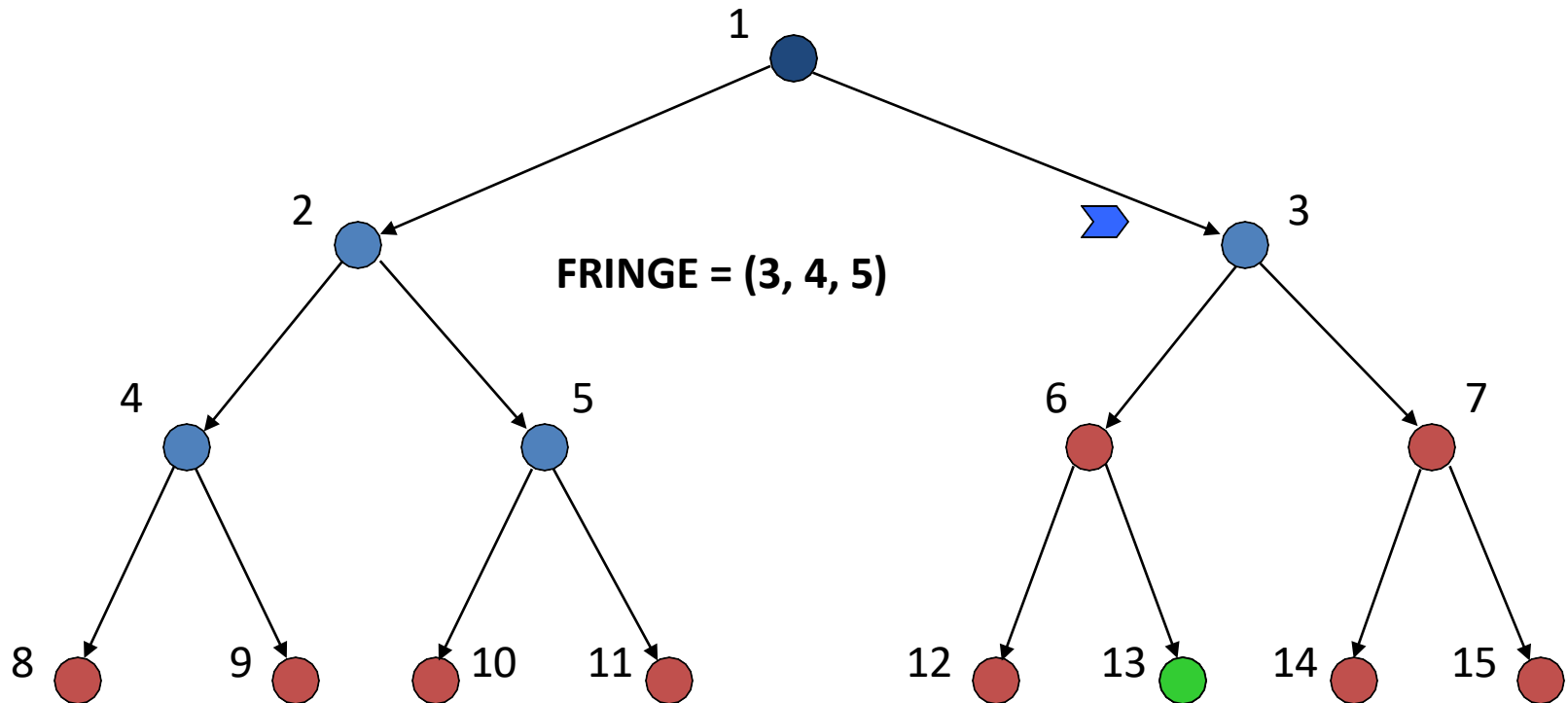
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



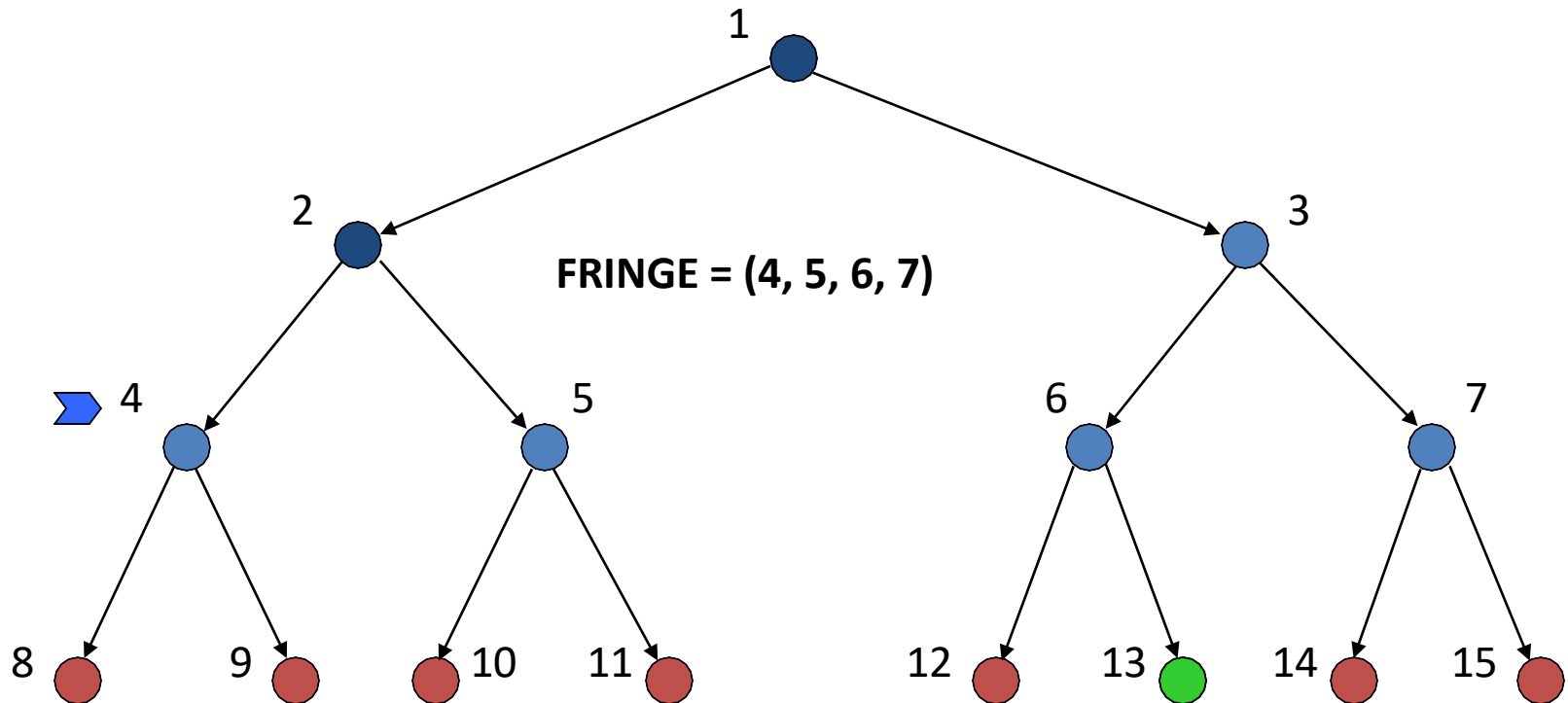
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



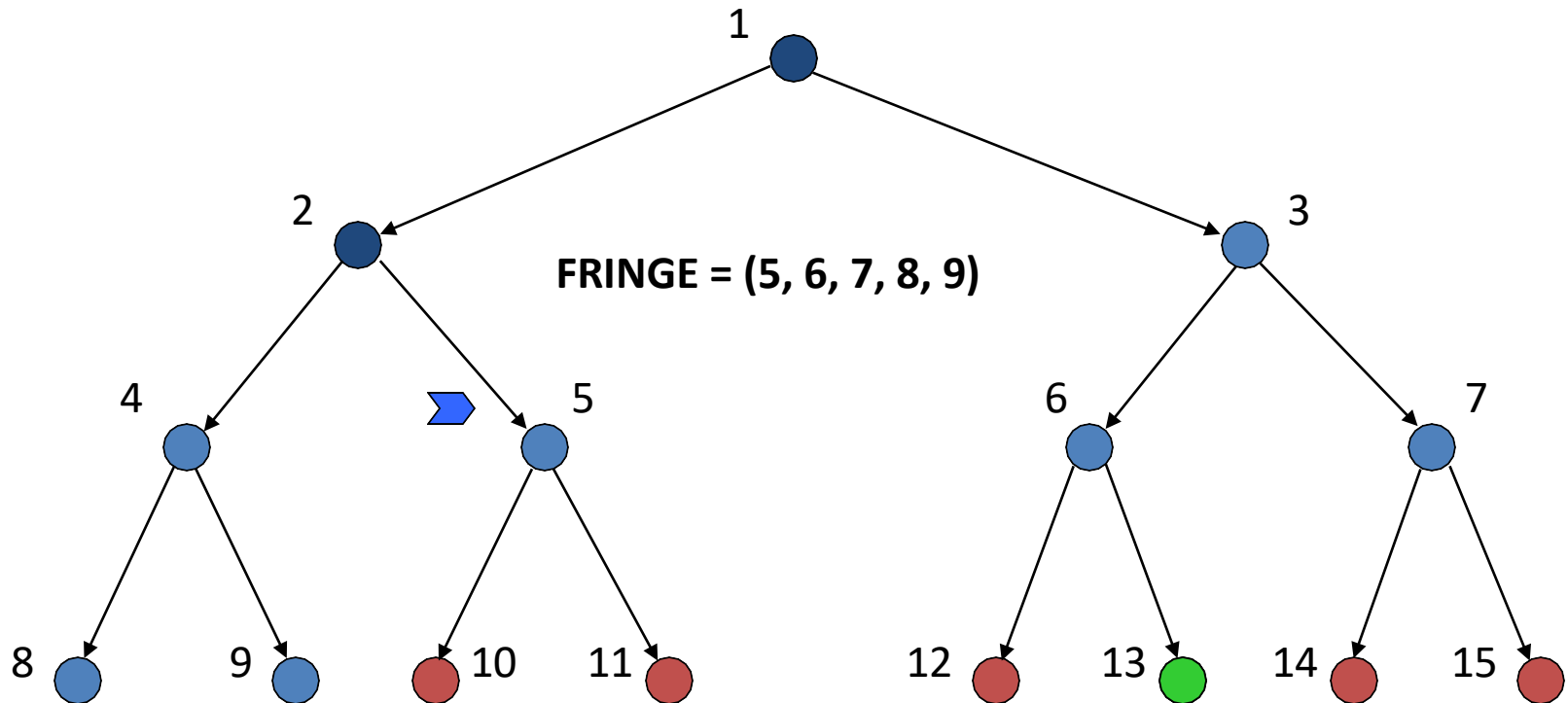
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



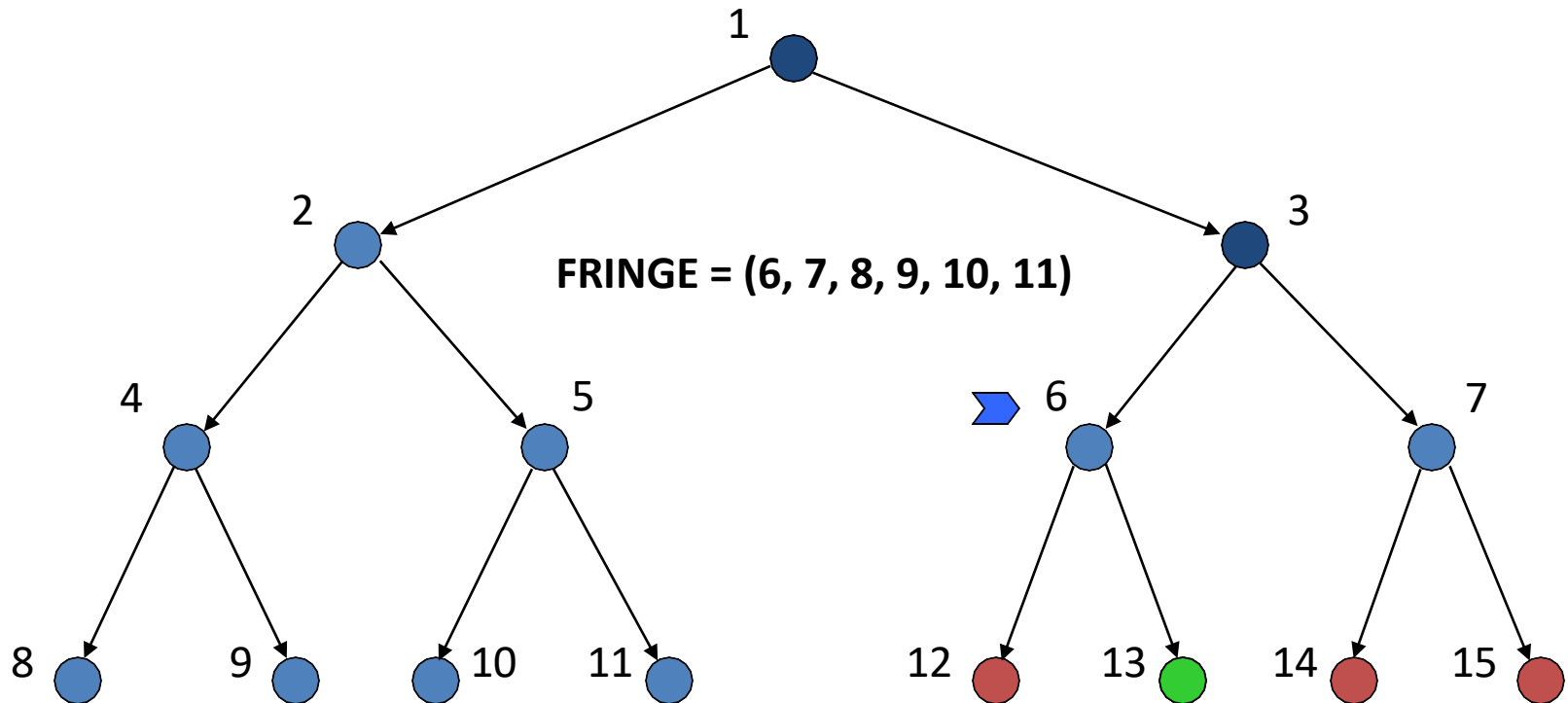
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



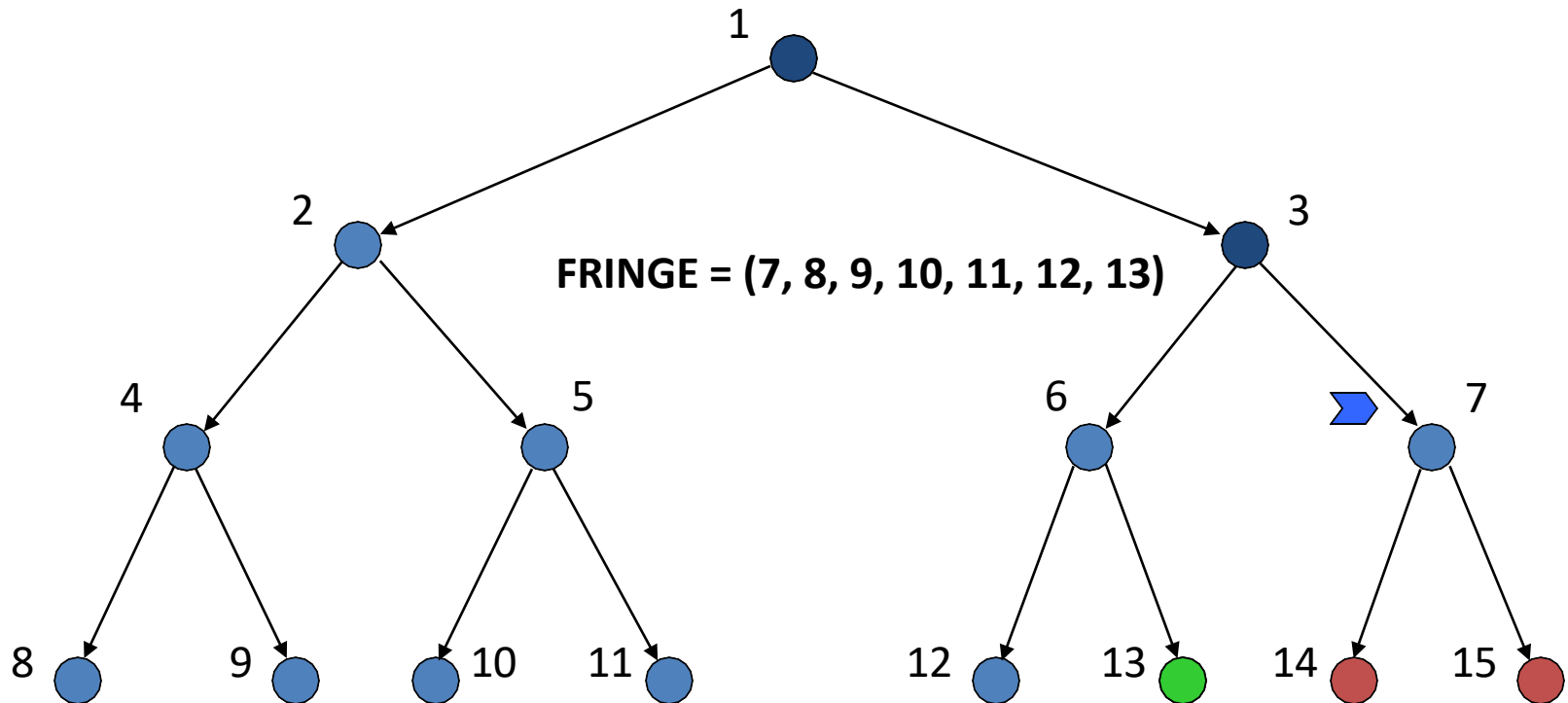
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



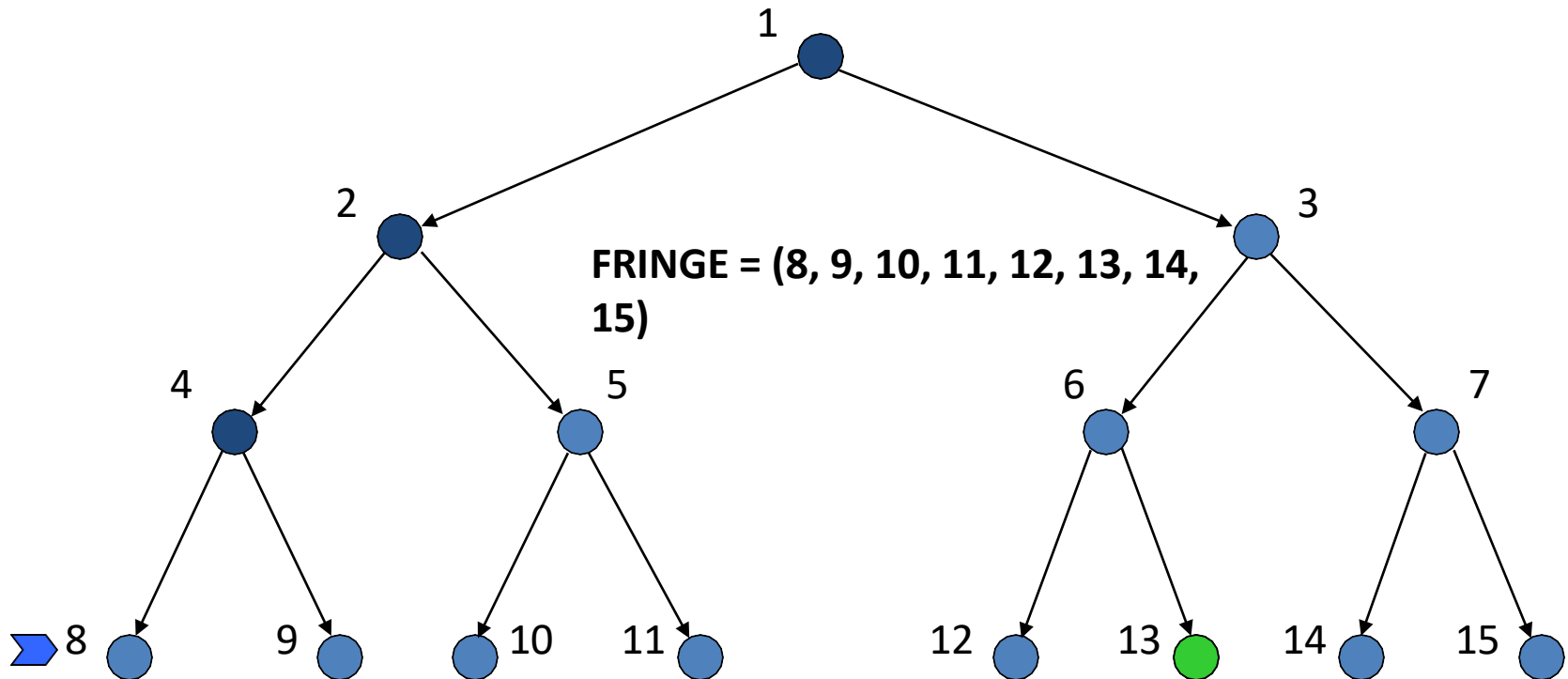
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



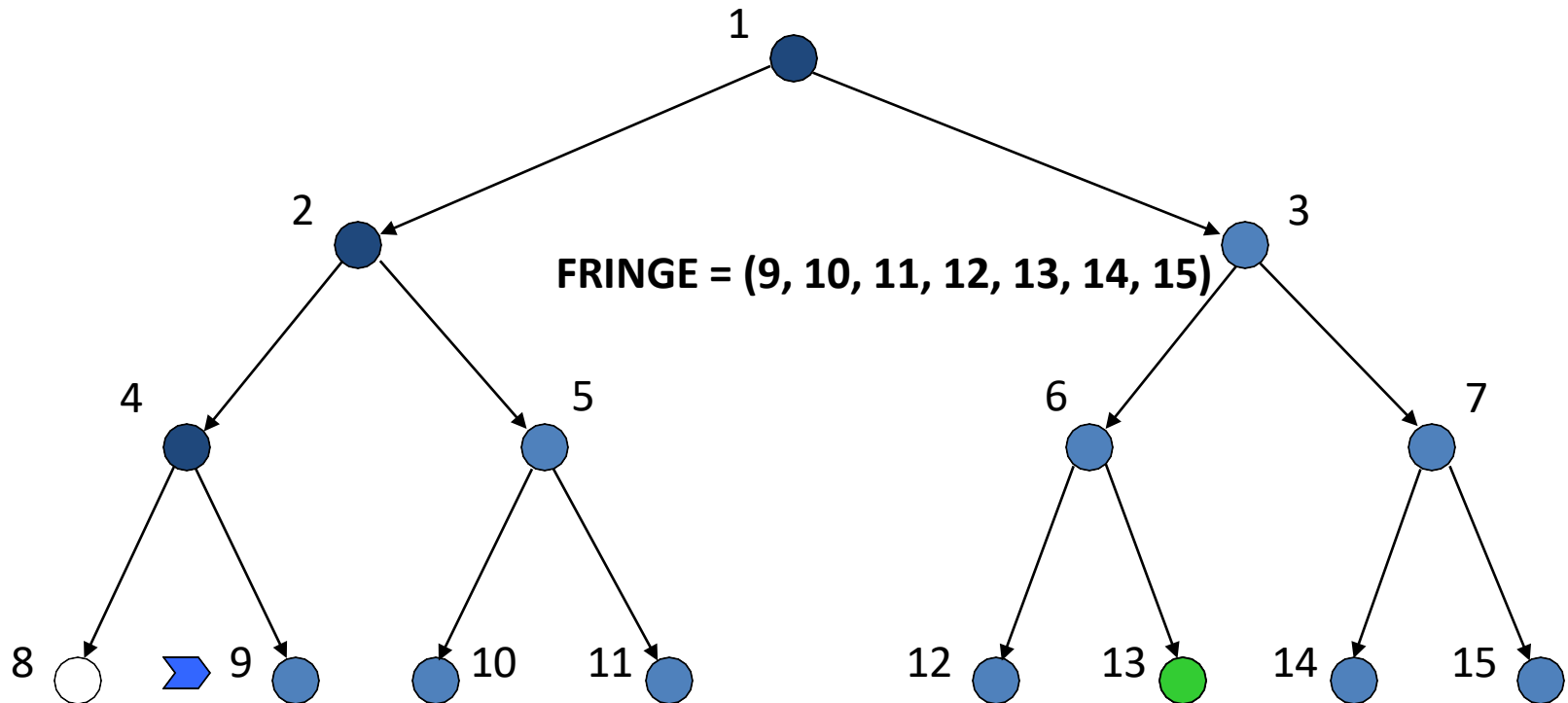
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



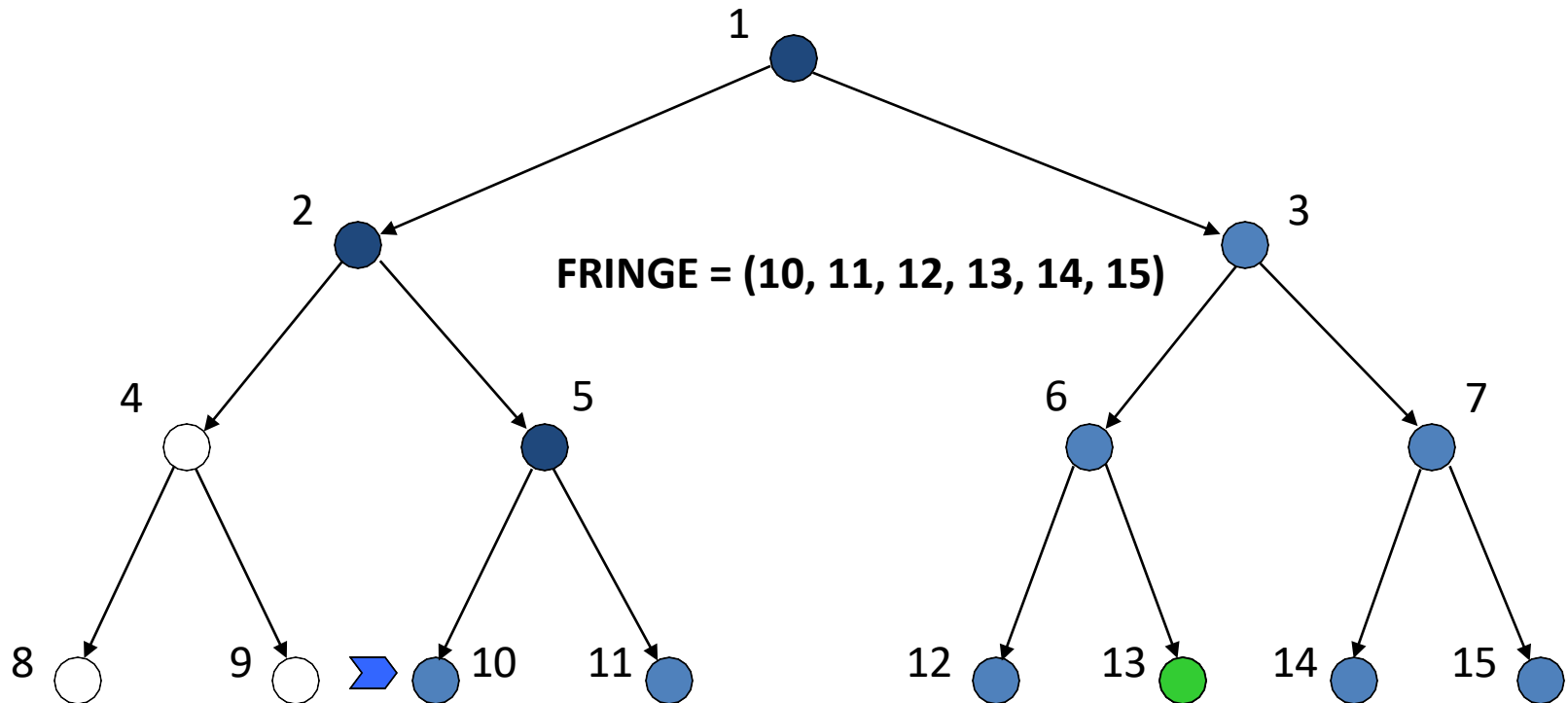
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



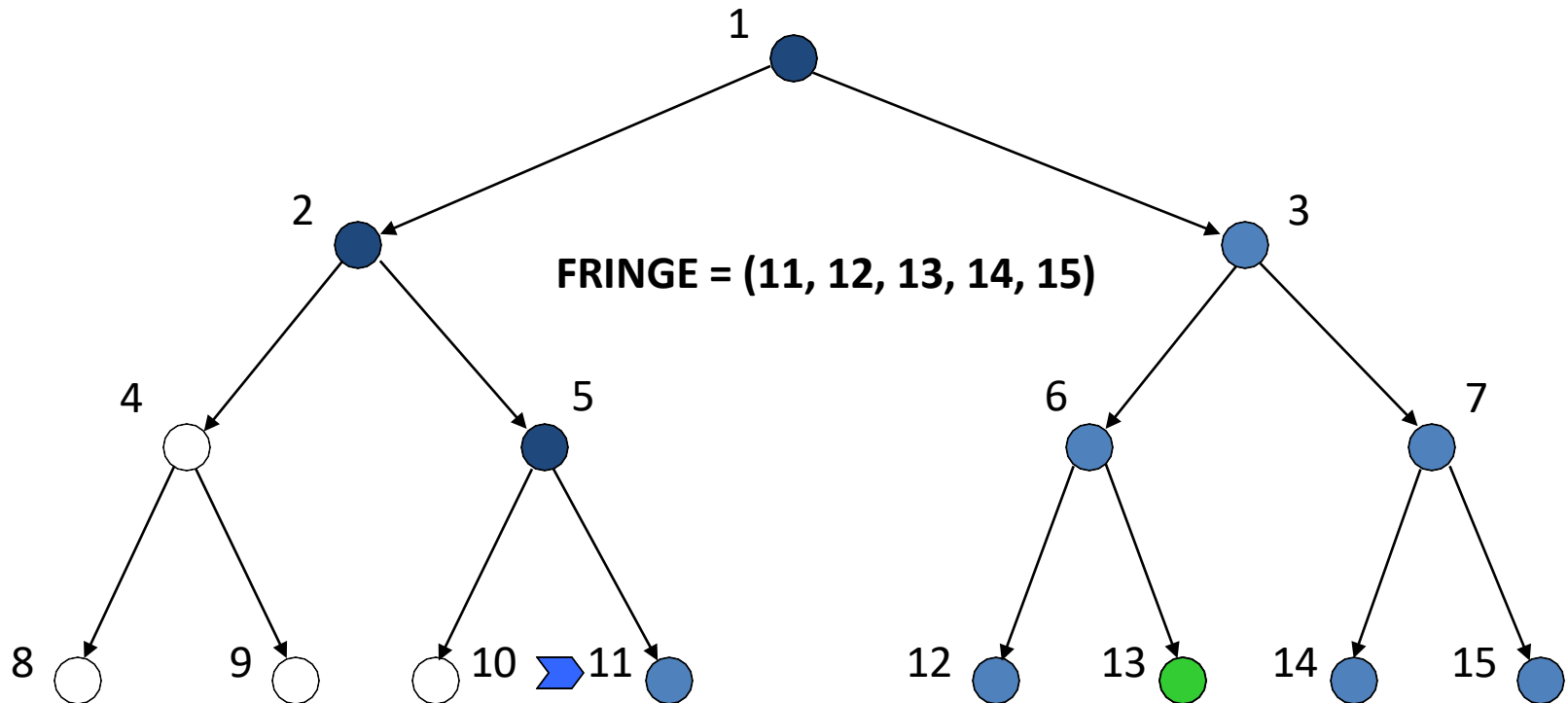
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



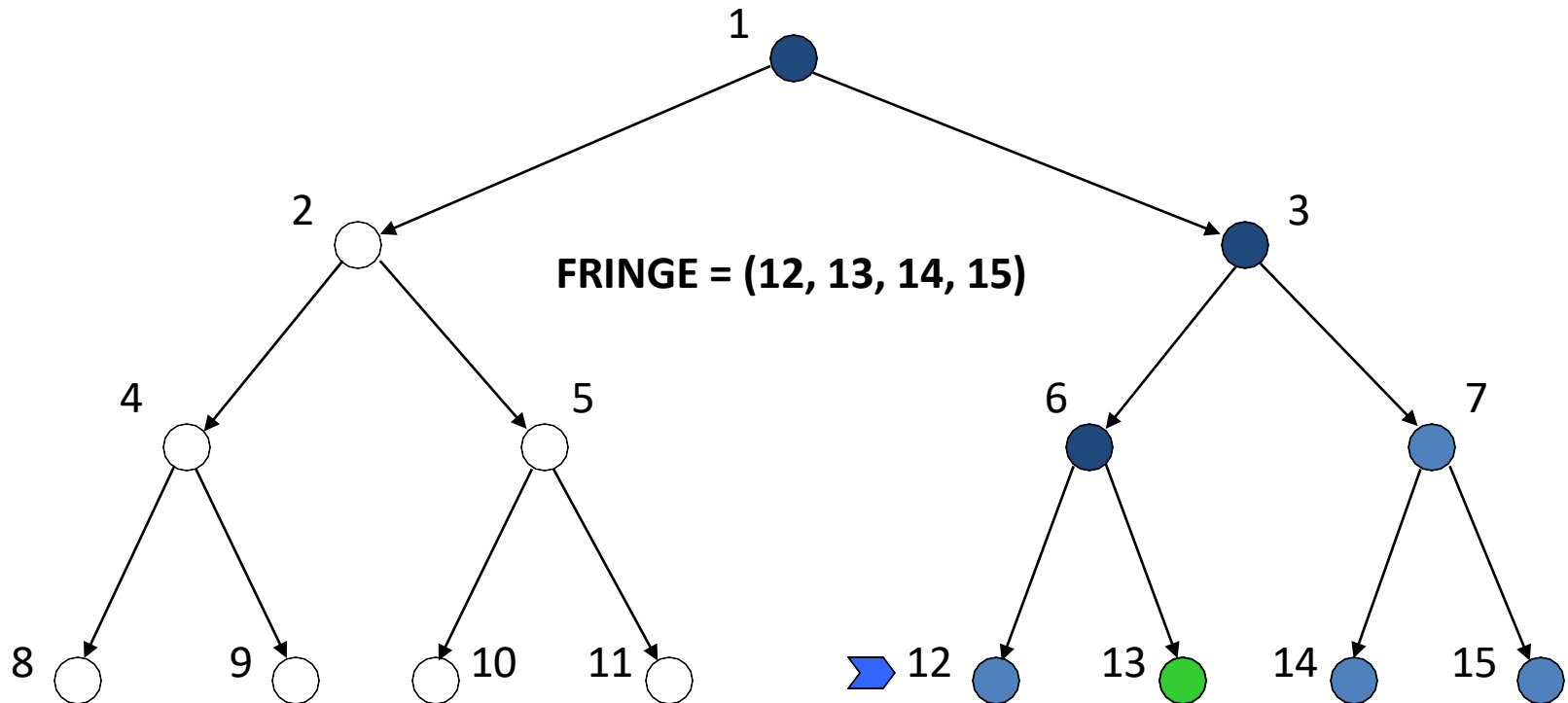
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.



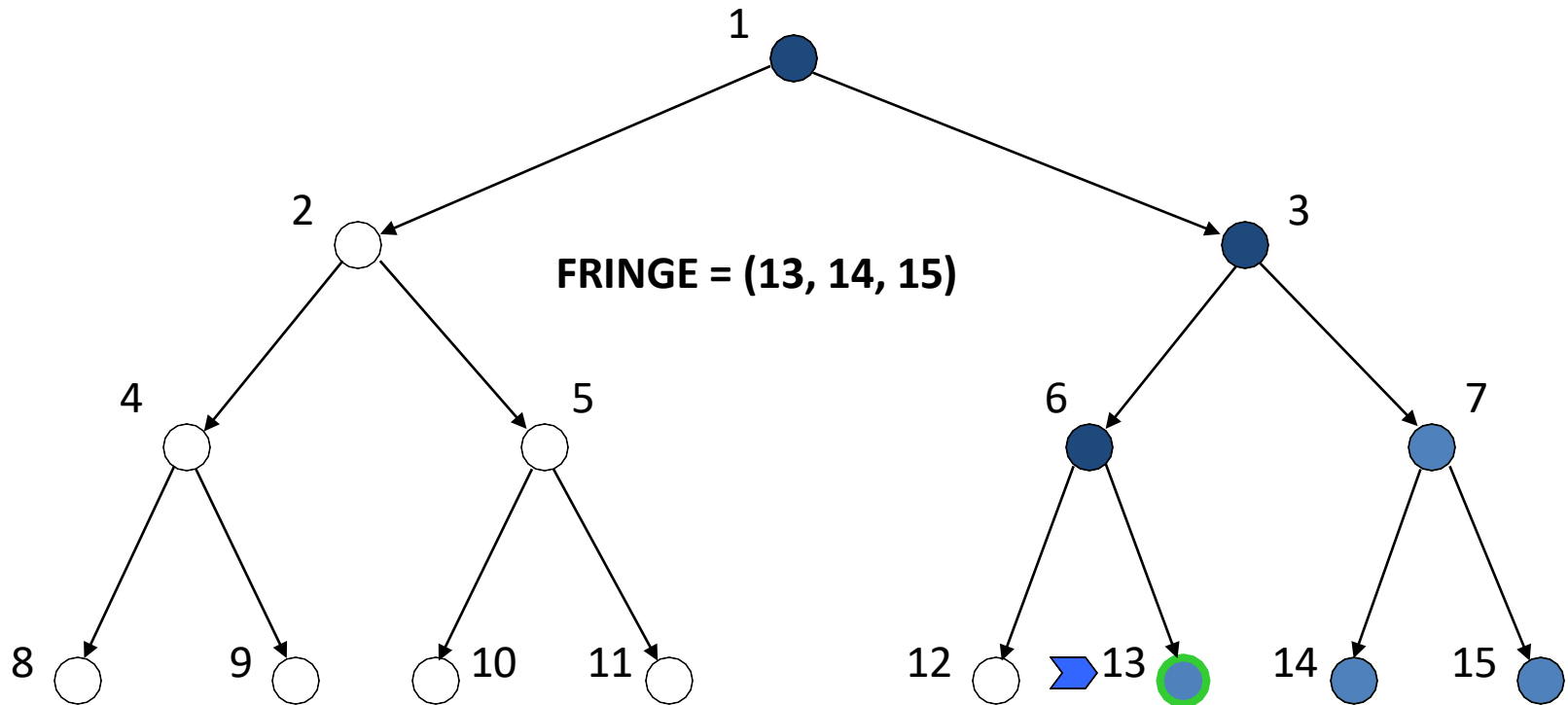
Breadth-First Strategy

New nodes are inserted **at the end** of the FRINGE.

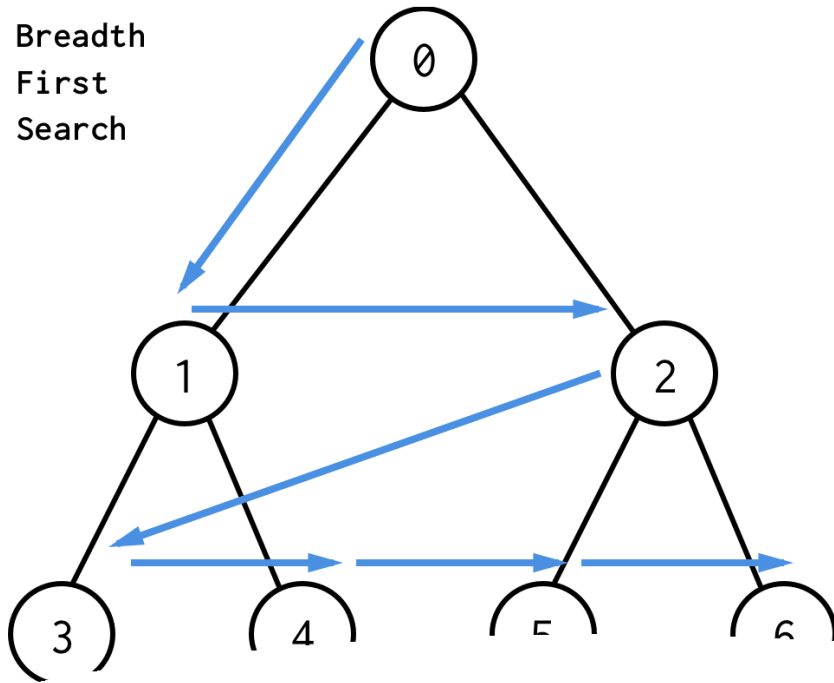


Breadth-First Strategy

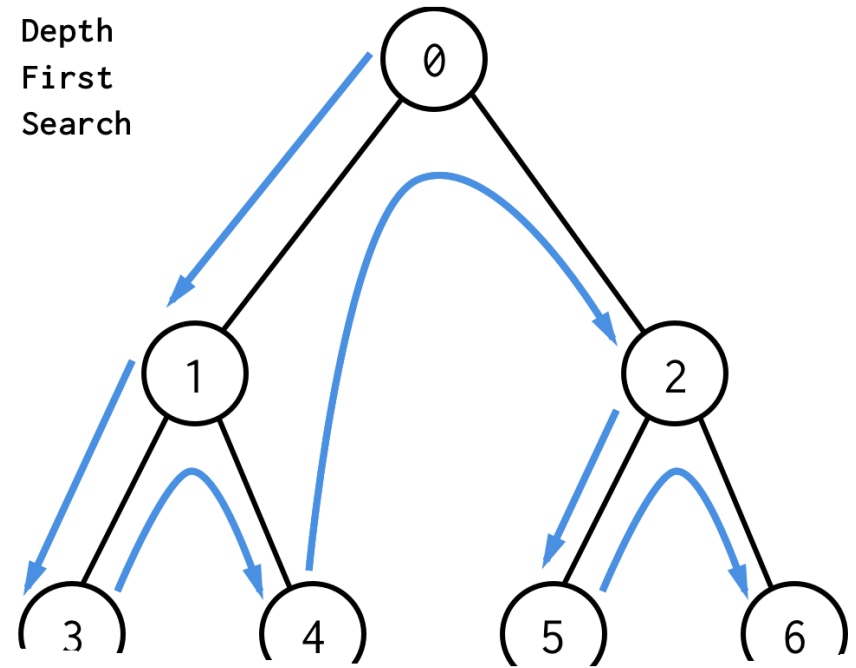
New nodes are inserted **at the end** of the FRINGE.



Breadth
First
Search



Depth
First
Search



Depth-First Strategy *versus* Breadth-First Strategy

In Depth-First Search, the FRINGE (*frontier*) is implemented using a ____(*data-structure*) ?

While in Breadth-First Search, the FRINGE (*frontier*) is implemented using a ____(*data-structure*) ?

Depth-Limited Strategy

Depth-first with **depth cutoff k** (*maximal depth below which nodes are not expanded*)

Three possible outcomes:

- Solution.
- Failure (*no solution*).
- **Cutoff** (*no solution within cutoff*).

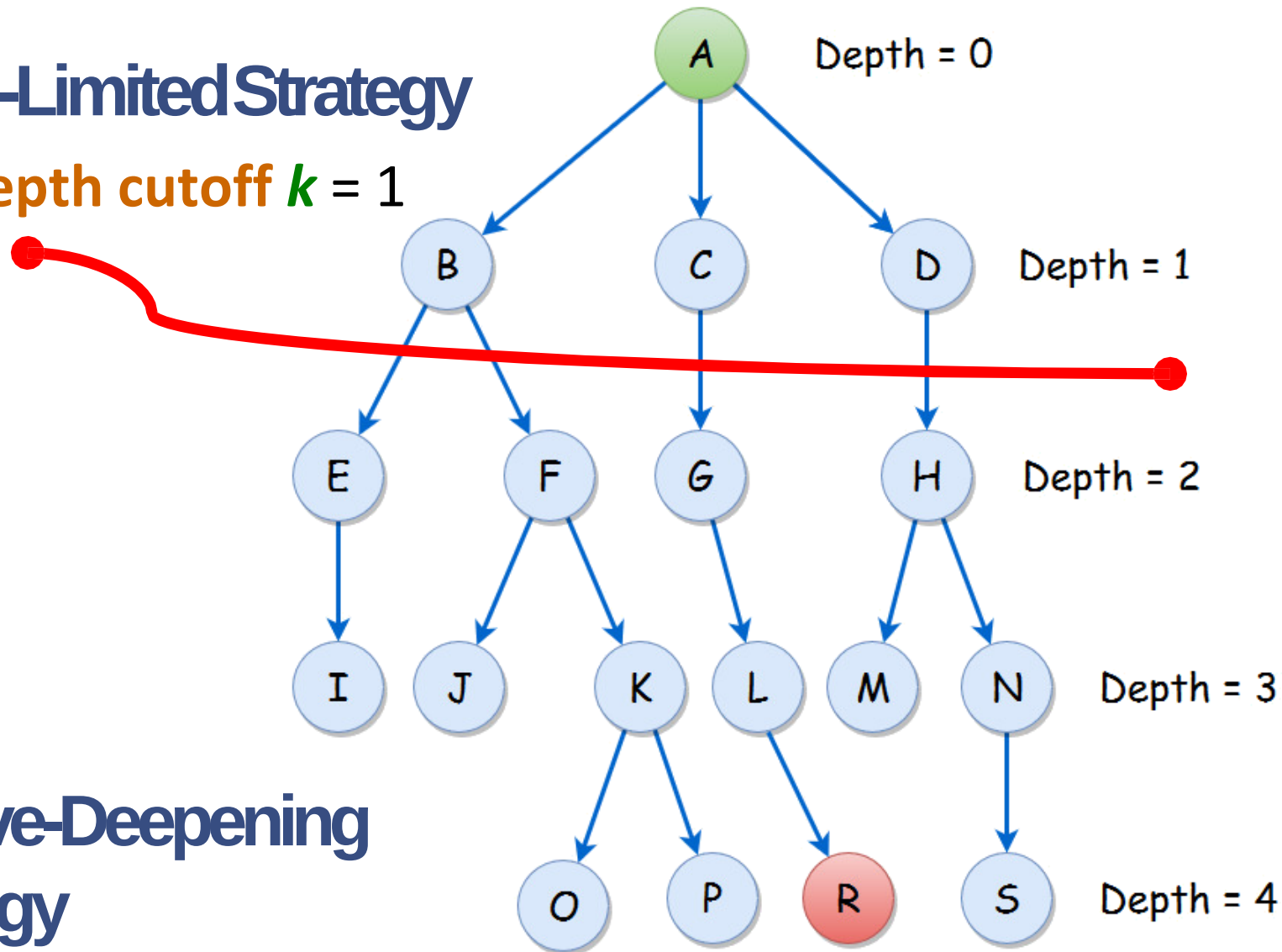
Iterative-Deepening Strategy

Repeat for **$k = 0, 1, 2, \dots$** :

- Perform depth-first with depth cutoff **k**

Depth-Limited Strategy

E.g., **depth cutoff** $k = 1$



Iterative-Deepening Strategy

E.g., Repeat for $k = 0$ to 4

- Perform depth-first with depth cutoff k

Comparing Depth, Breadth, & Iterative Deepening Search Algorithms

- Breadth-first and iterative deepening guarantee shortest solution.
- Breadth-first: high space complexity.
- Depth-first: low space complexity, but may search well below solution depth.
- Iterative deepening: best performance in terms of orders of complexity.

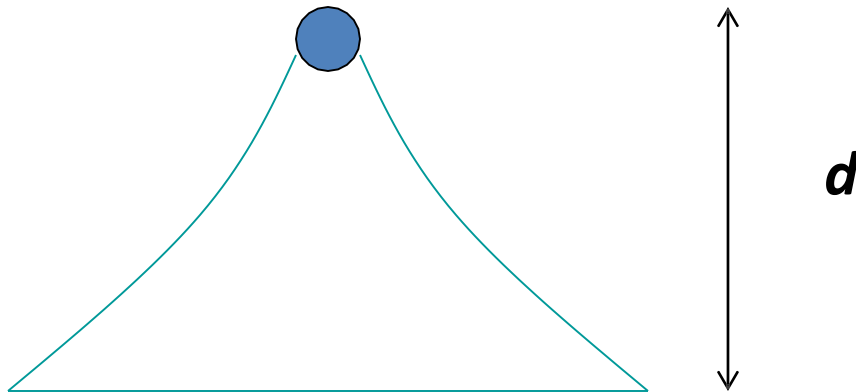
Bidirectional Search

Bidirectional search is a graph search algorithm that finds a shortest path from an initial start state to a goal state in a directed graph. It **runs two simultaneous searches**: one forward from the initial state, and one backward from the goal, stopping when the two meet.

The reason for this approach is that **in many cases it is faster**: for instance, in a simplified model of search problem complexity in which both searches expand a tree with branching factor b , and the distance from start to goal is d , each of the two searches has complexity $O(b^{d/2})$ (in Big O notation), and the sum of these two search times is much less than the $O(b^d)$ complexity that would result from a single search from the beginning to the goal.

Complexity of Basic Search Algorithms

- For simpler analysis consider state-space as a tree
- Uniform branching b
- Solution at depth d



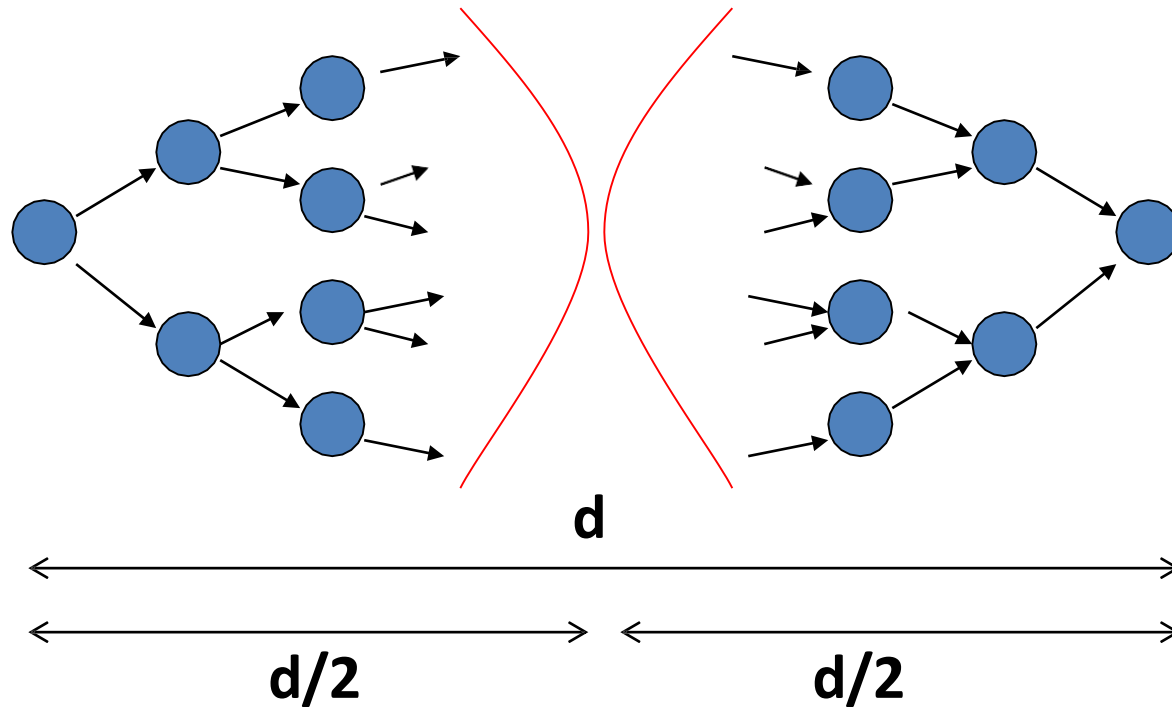
Number of nodes at level d : b^d

Bidirectional Search

Complexity of Bidirectional Search

Consider the following case:

- forward and backward branching both b , uniform ..



$$\text{Time} \sim b^{d/2} + b^{d/2} < b^d$$

Repeated States

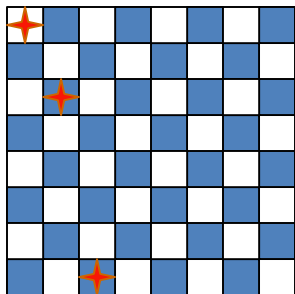
No

Few

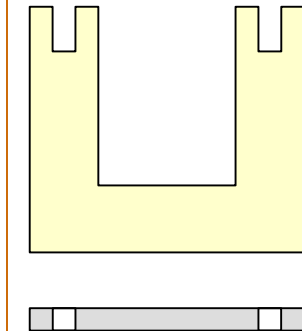
Many

search tree is finite

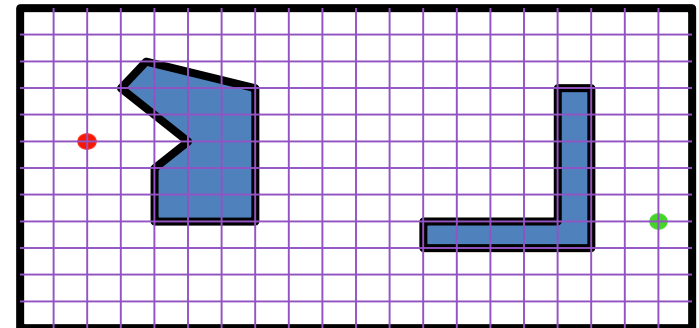
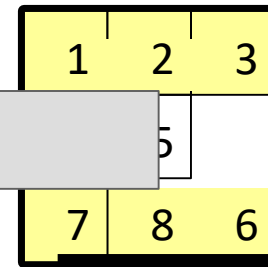
search tree is infinite



8-queens



assembly
planning



8-puzzle and robot navigation

Avoiding Repeated States

Requires comparing state descriptions.

For example, Breadth-first strategy:

- Keep track of all generated states.
- If the state of a new node already exists, then discard the node.

Avoiding Repeated States

Depth-first strategy:

Solution 1:

- Keep track of all states associated with nodes in current path
- If the state of a new node already exists, then discard the node

Solution 2:

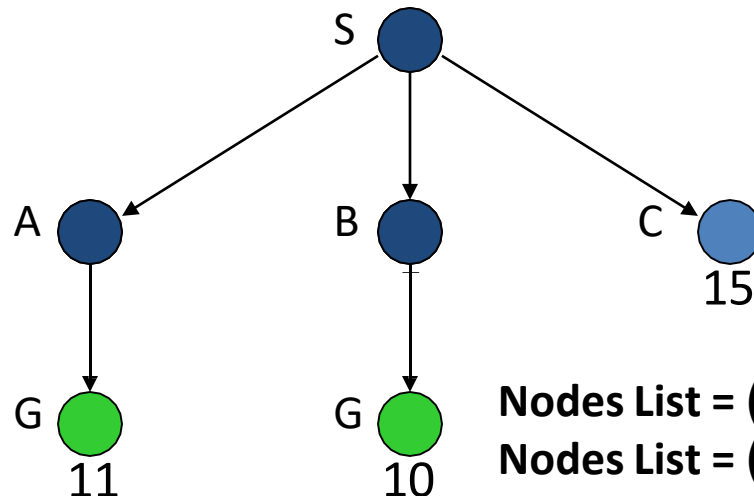
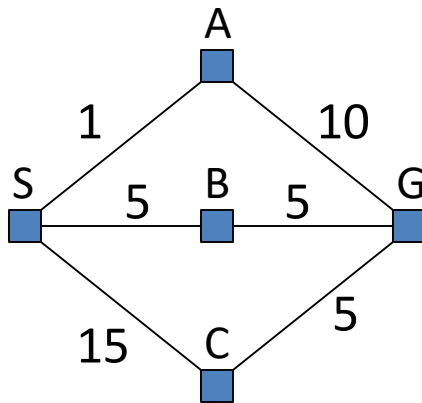
- Keep track of all states generated so far
- If the state of a new node has already been generated, then discard the node

Uniform-Cost Strategy

- Each step has some cost $\geq \epsilon > 0$.
- The cost of the path to each fringe node N is

$$g(N) = \sum \text{costs of all steps.}$$

- The goal is to generate a solution path of minimal cost.
- The queue FRINGE is sorted in increasing cost.



Nodes List = (S0)

Nodes List = (A1, B5, C15)

Nodes List = (B5, G11, C15)

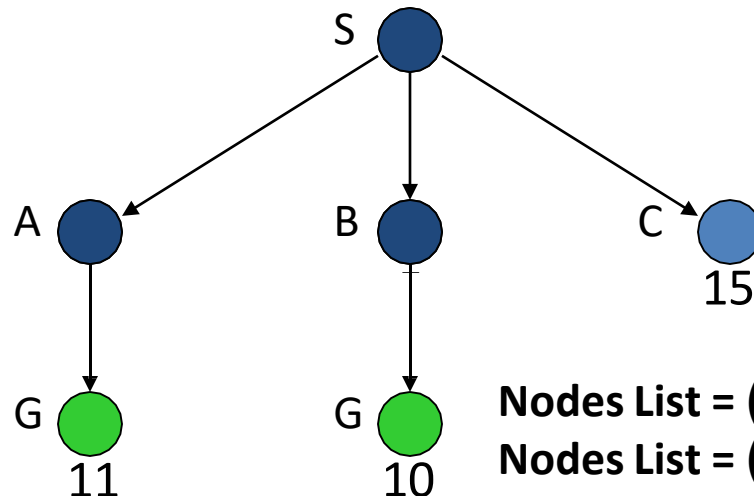
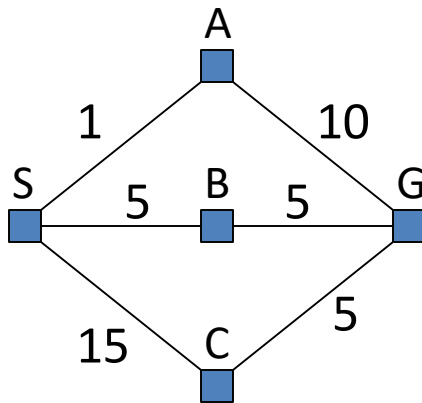
Nodes List = (G10, G11, C15)

Uniform-Cost Strategy

- Each step has some cost $\geq \epsilon > 0$.
- The cost of the path to each fringe node N is

$$g(N) = \sum \text{costs of all steps.}$$

- The goal is to generate a solution path of minimal cost.
- The queue FRINGE is sorted in increasing cost.



Nodes List = (S0)

Nodes List = (A1, B5, C15)

Nodes List = (B5, G11, C15)

Nodes List = (G10, G11, C15)

Informed Search

[Heuristic Search]



Best-FirstSearch

- Define a function:

$$f : \text{node } N \rightarrow \text{real number } f(N)$$

called the **evaluation function**, whose value depends on the contents of the state associated with **N**.

- Order the nodes in the fringe in increasing values of **f(N)**.
- **f(N)** can be any function you want, but will it work?

Heuristic Function

Function $h(N)$ that estimate the cost of the cheapest path from node N to goal node.

Example: 8-puzzle

5		8
4	2	1
7	3	6

N

1	2	3
4	5	6
7	8	

goal

$$h(N) = \text{number of misplaced tiles} \\ = 6$$

Heuristic Function

Function $h(N)$ that estimate the cost of the cheapest path from node N to goal node.

Example: 8-puzzle

5		8
4	2	1
7	3	6

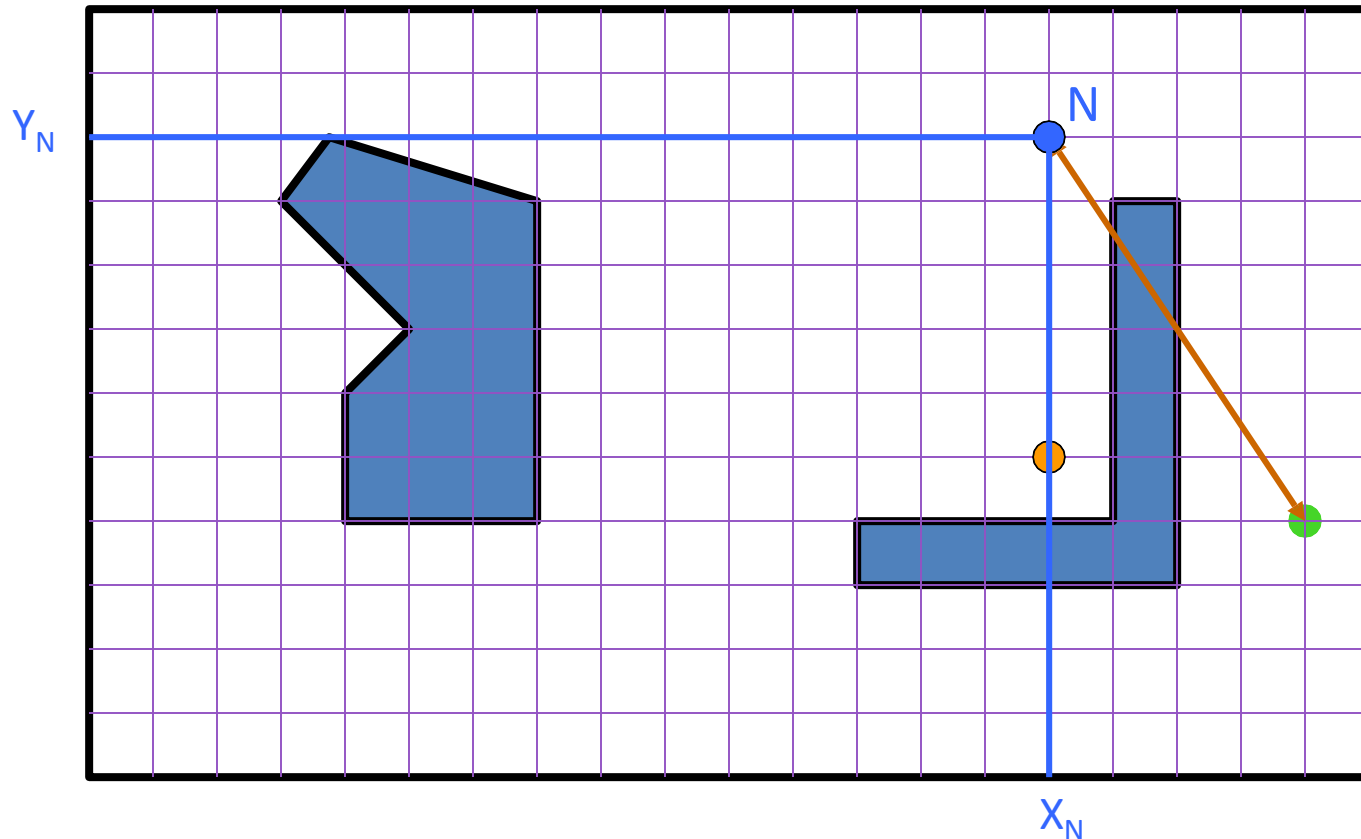
N

1	2	3
4	5	6
7	8	

goal

$$\begin{aligned} h(N) &= \text{sum of the distances of} \\ &\quad \text{every tile to its goal position} \\ &= 2 + 3 + 0 + 1 + 3 + 0 + 3 + 1 \\ &= 13 \end{aligned}$$

Robot Navigation



$h(N)$ = Straight-line distance to the goal
 $= [(X_g - X_N)^2 + (Y_g - Y_N)^2]^{1/2}$

Examples of Evaluation function

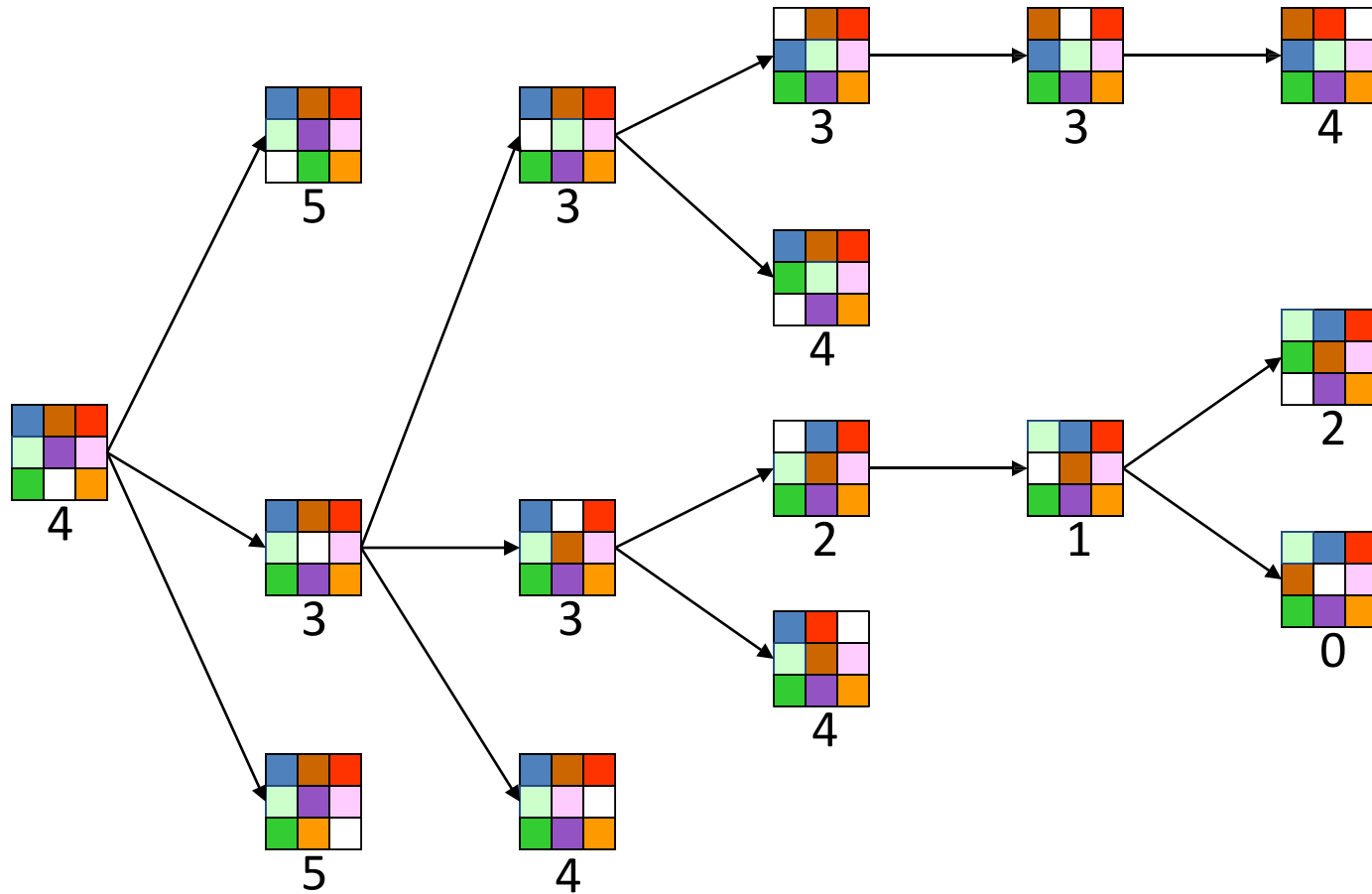
Let $g(N)$ be the cost of the best path found so far between the initial node and N

$f(N) = h(N) \rightarrow$ greedy best-first

$f(N) = g(N) + h(N)$

8-Puzzle

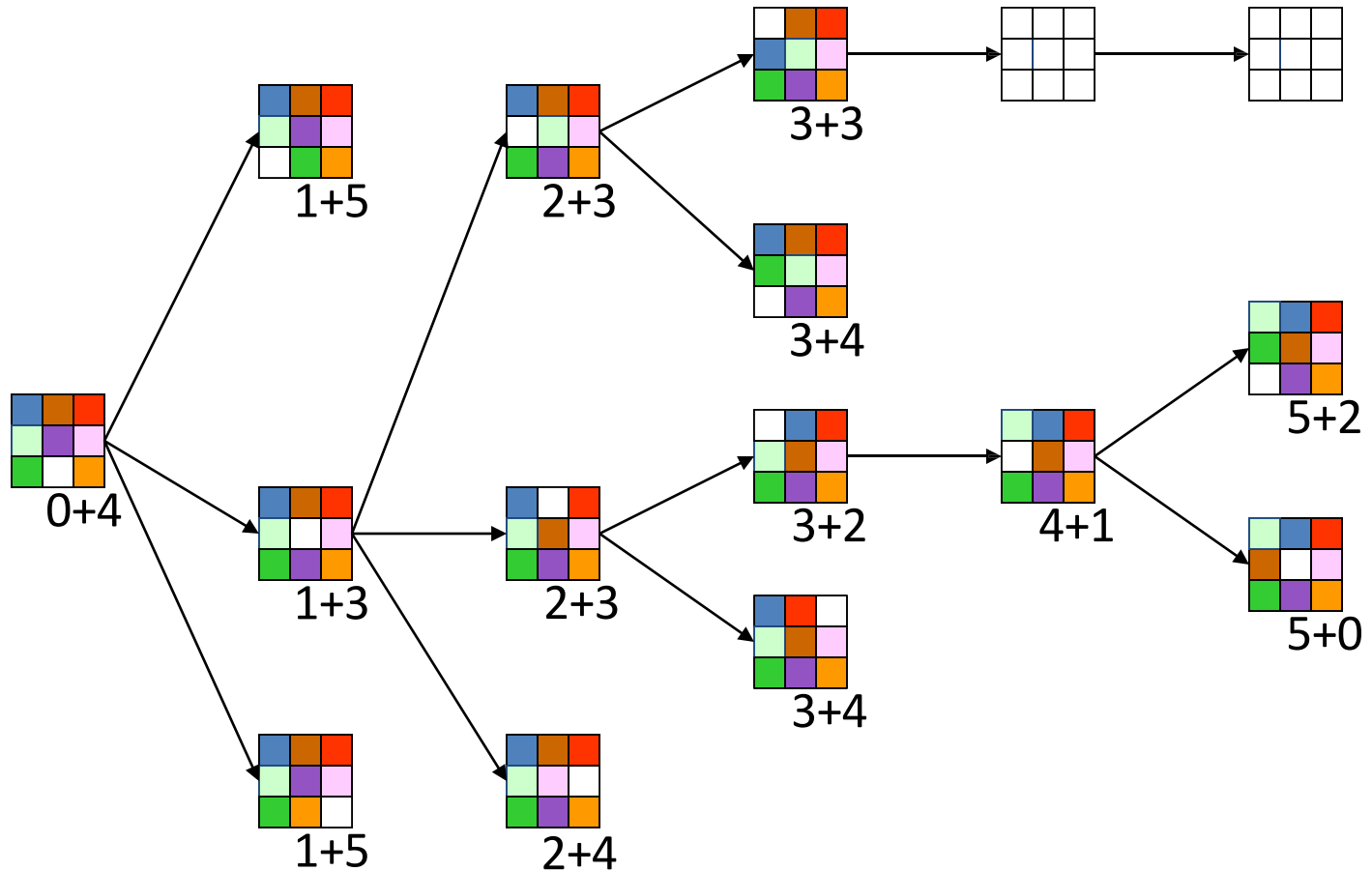
$f(N) = h(N) = \text{number of misplaced tiles}$



8-Puzzle

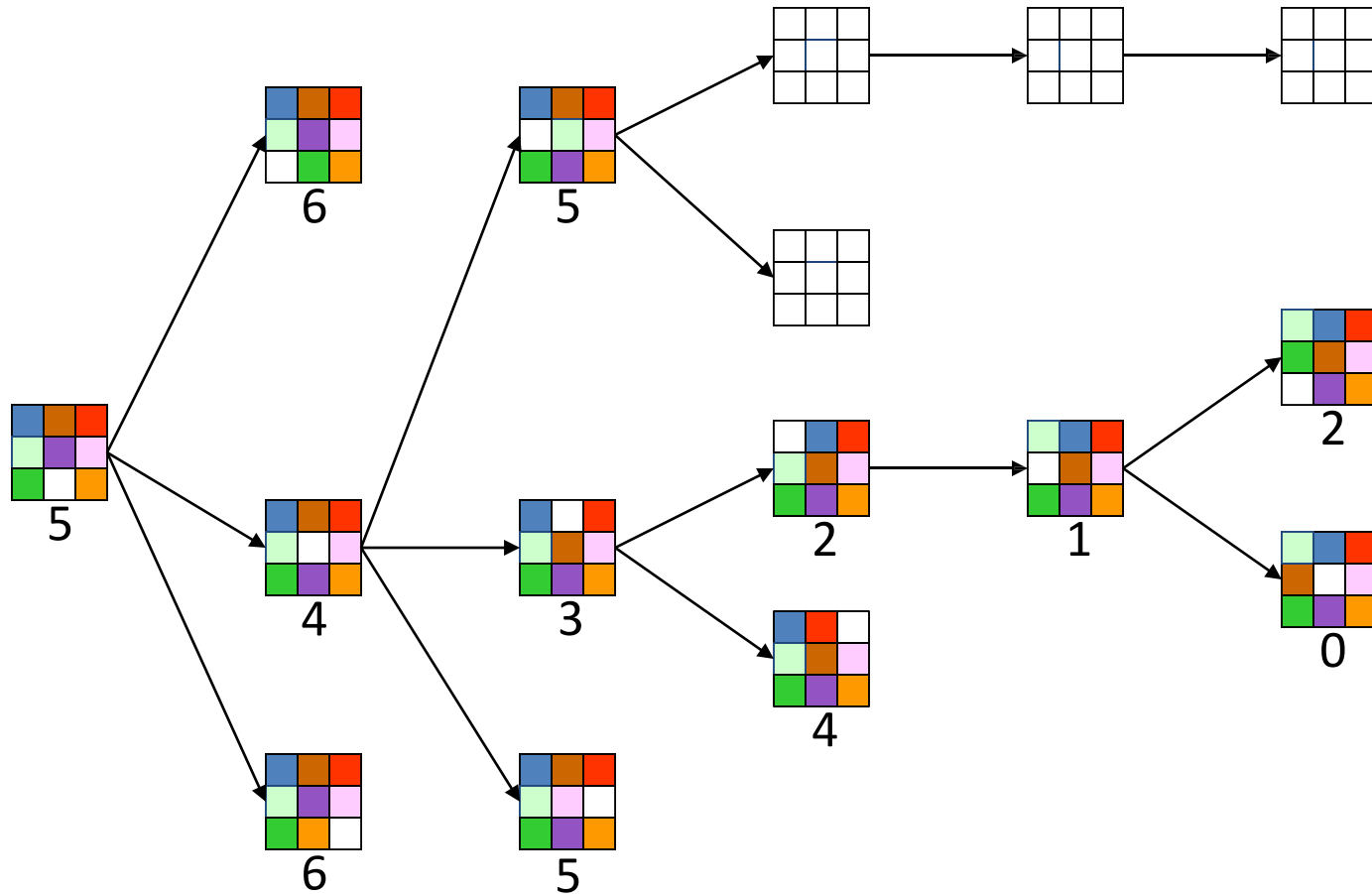
$$f(N) = g(N) + h(N)$$

with $h(N)$ = number of misplaced tiles



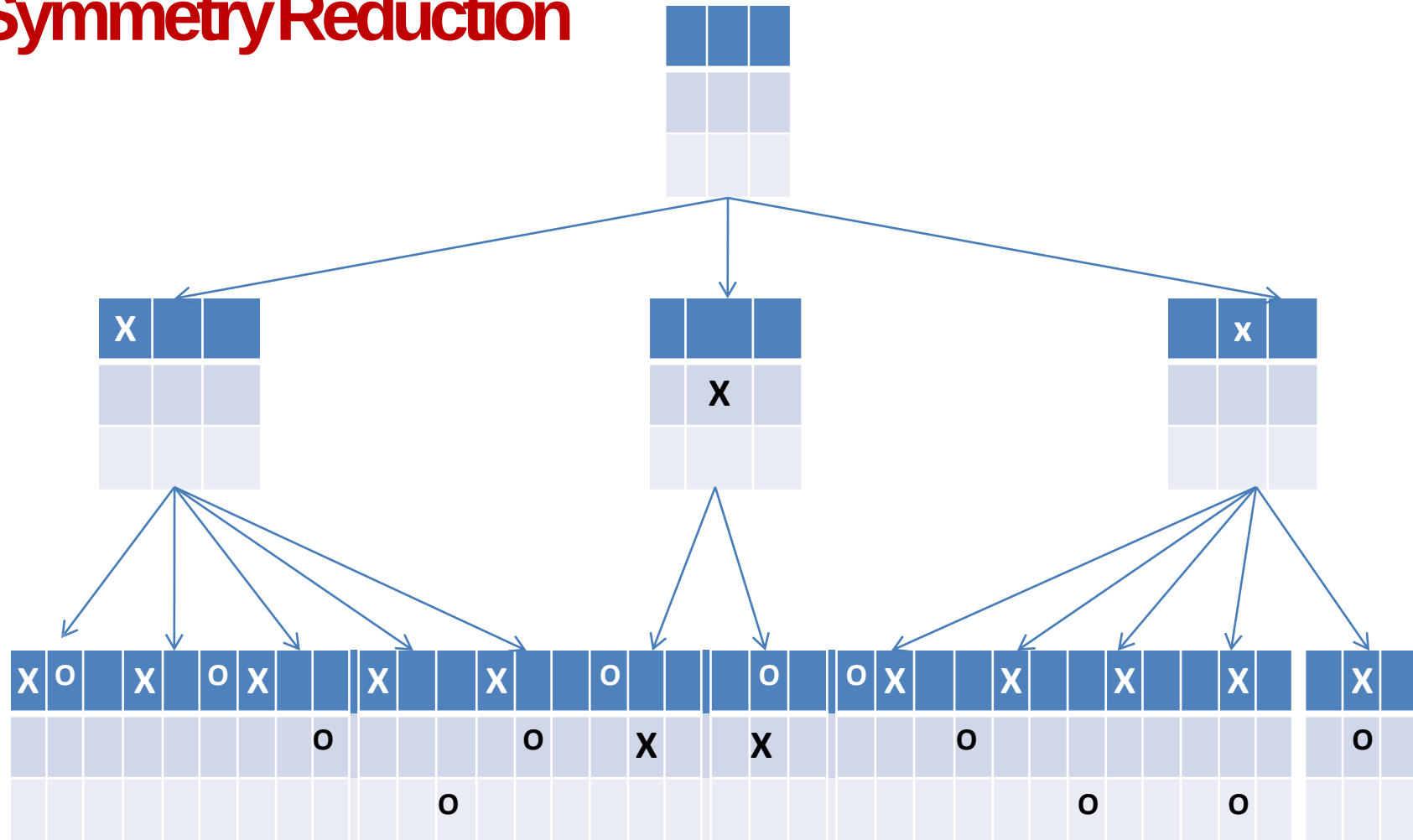
8-Puzzle

$$f(N) = h(N) = \sum \text{distances of tiles to goal}$$



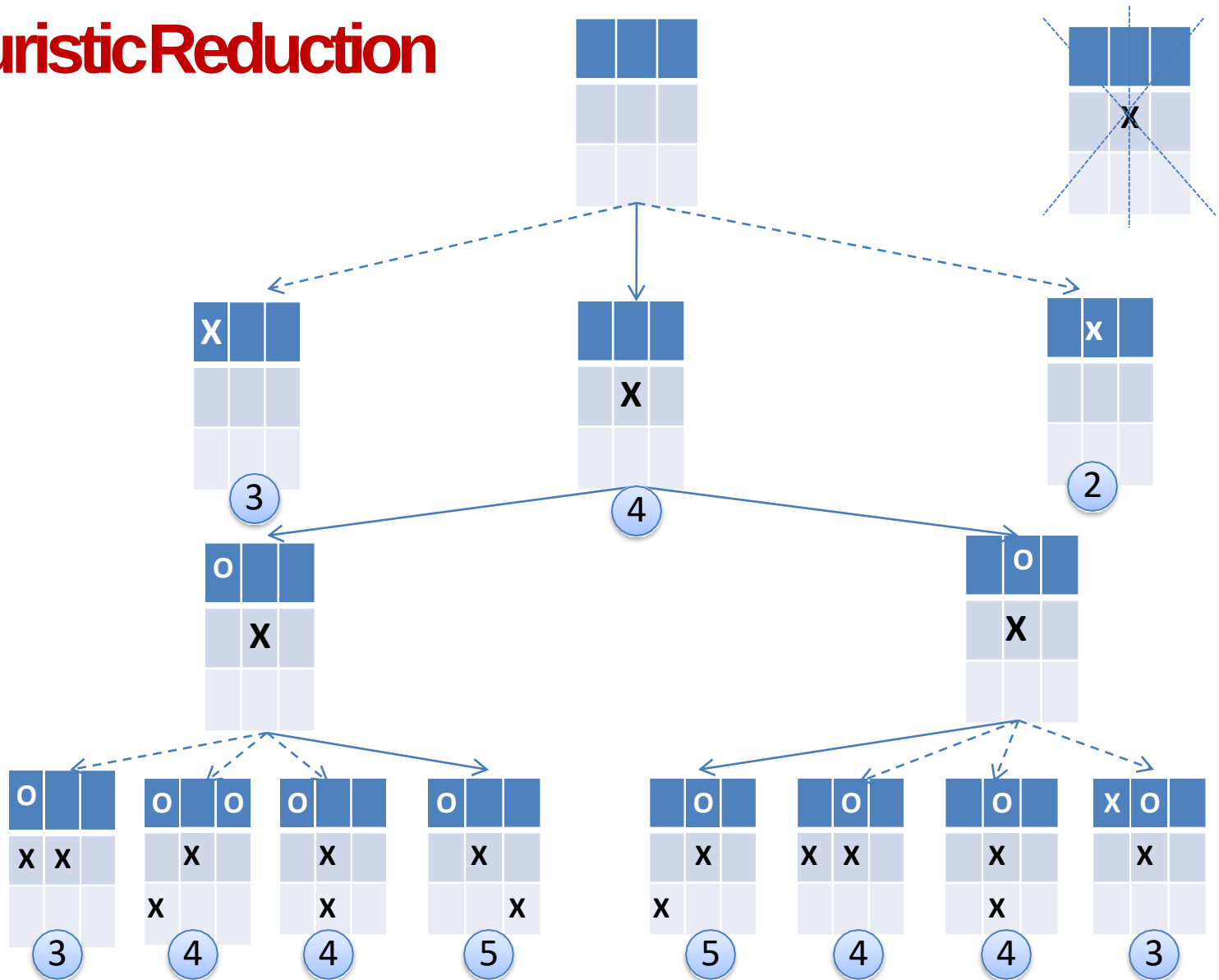
More on Heuristic Search & Functions

Symmetry Reduction



More on Heuristic Search & Functions

Heuristic Reduction



More on Heuristic Search & Functions

Hill Climbing Strategy

- “Method of blind mountain climbers to go uphill along the steepest possible path until no farther to go”
- Expand to the current state and evaluate its children. The best child is selected for further expansion until reaching a state that is better than any of its children.
- It fails to find a solution in case of the reaching state is not a goal but just a local maximum state.
- The strategy can be used effectively if the evaluation function is sufficiently informative to distinguish between local and global maximum.

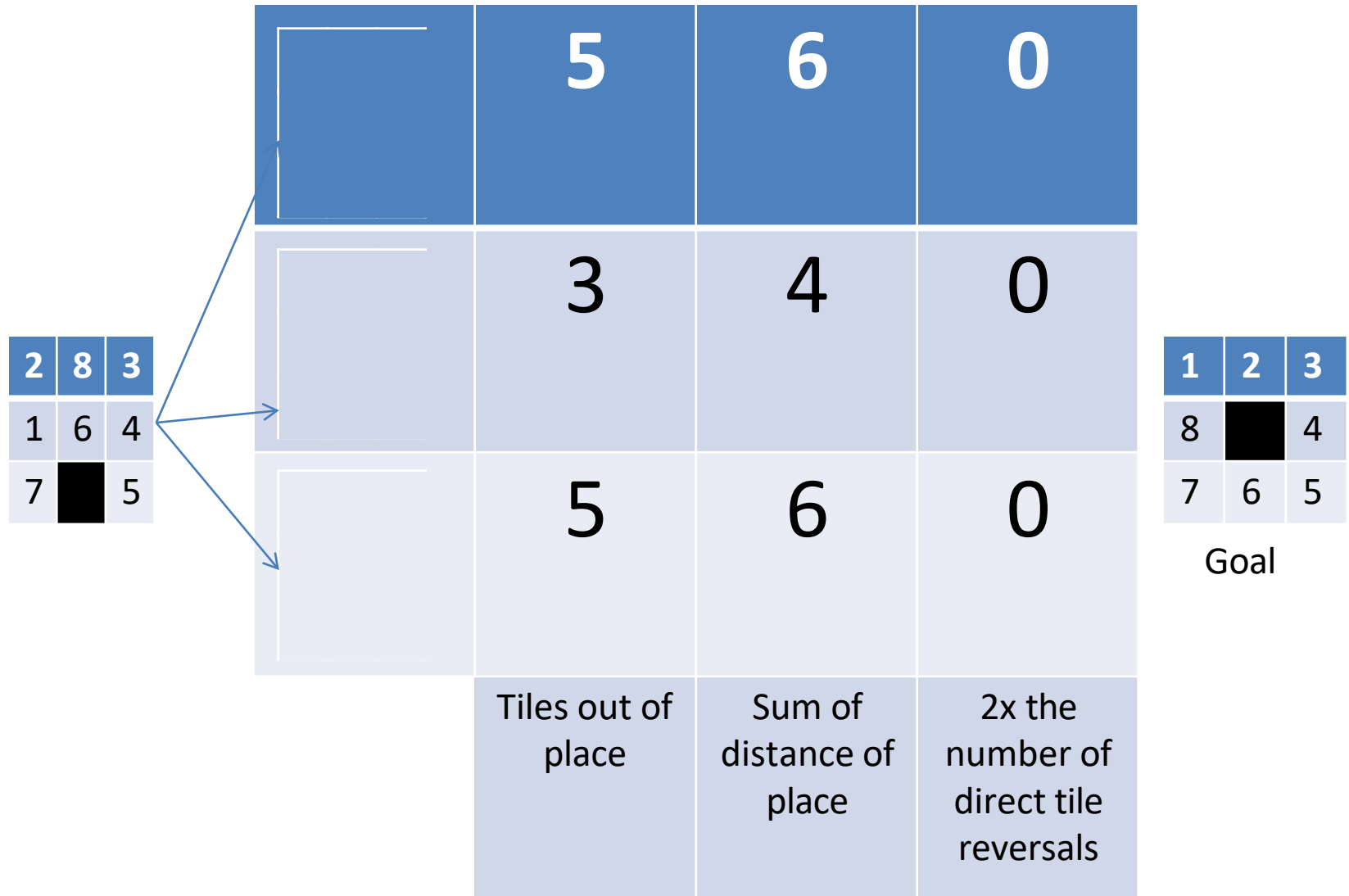
More on Heuristic Search & Functions

Best-First Search

- Use open list to keep track of the current edge of the search and closed list to record states already visited.
- Order the states on open list according to some heuristic estimate of their closeness to a goal and the most promising state is considered in each iteration.
- The best state to be examined in each iteration may be from any level of the state space.
- Open sorted list is represented as a **priority queue**.

More on Heuristic Search & Functions

8-Puzzle Heuristics



More on Heuristic Search & Functions

8-Puzzle Heuristics

$g(n)=0$

$g(n)=1$

$g(n)=2$

$g(n)=3$

$g(n)=4$

$g(n)=5$

2	8	3
1	6	4
7		5

Start

$f(a)=4$

2	8	3
1	6	4
	7	5

$f(b)=6$

2	8	3
1		4
7	6	5

$f(c)=4$

2	8	3
1	6	4
7	5	

$f(d)=6$

2	8	3
	1	4
7	6	5

$f(e)=5$

2		3
1	8	4
7	6	5

$f(f)=5$

2	8	3
1	4	
7	6	5

$f(g)=6$

1	2	3
8		4
7	6	5

Goal

$$f(n) = h(n) + g(n)$$

Thanks! ... *Questions?*